



Addis Ababa University
College of Natural Science

Design and Evaluation of a Security-Integrated Vibe Coding Framework for Node.js
Microservices.

Fantahun Bishaw Hailu

A Thesis Submitted to the Department of Computer Science in Partial Fulfillment for the Degree
of Master of Science in Computer Science

Addis Ababa, Ethiopia
Feb 28, 2026

Addis Ababa University
College of Natural Sciences

Fantahun Bishaw Hailu

Advisor: Dr. Ayalew Belay

This is to certify that the thesis prepared by *Fantahun Bishaw Hailu*, titled: *Design and Evaluation of a Security-Integrated Vibe Coding Framework for Node.js Microservices* and submitted in partial fulfillment of the requirements for the Degree of Master of Science in Computer Science complies with the regulations of the University and meets the accepted standards with respect to originality and quality.

Signed by the Examining Committee:

Name	Signature	Date
Advisor:	_____	_____
Examiner:	_____	_____
Examiner:	_____	_____

Abstract

Dedication

Acknowledgements

Table of Contents

1. Introduction	1
2. Literature Review	7
3. Related Work	17
4. The Proposed Solution	24
5. Experimentation/Prototype and Evaluation	58
6. Conclusions	58
References	59
Annexes.....	61
Annex A: Title of the annex.....	61
Annex B: Title of the annex.....	61

Acronyms and Abbreviations

1. Introduction

The introduction content will be here

Below is a polished **Chapter 1: Introduction** draft, kept compact so it can fit roughly within **two pages** in a standard thesis layout. It follows the chapter pattern indicated in the guideline and your template, namely **Background, Motivation, Statement of the Problem, Objectives, Methods, Scope and Limitations, Application of Results, and Organization of the Rest of the Thesis**. 1
2 3

Keep the citation numbers **exactly in this order** if you want Chapter 1 to remain consistent with the final references list.

Chapter 1

Introduction

This chapter introduces the context, motivation, problem, objectives, method, scope, and expected value of the study. It frames the research within recent advances in AI-assisted software engineering and explains why security must be treated as a first-class concern when large language models are used to generate backend code for Node.js microservices. 4 5

1.1 Background

Large language models are changing the way software is produced. Recent research shows that AI systems are no longer used only for autocomplete or short code snippets, but increasingly for code generation, repair, summarization, and repository-level software engineering tasks [1], [2]. 6 7 A newer development style, often described as *vibe coding*, pushes this transition further by allowing developers to express intent in natural language while AI assistants generate or revise implementation details [3]. 8

The appeal of this approach is largely driven by speed. Controlled evidence shows that AI-assisted programming can reduce development time in some tasks, including a study in which developers completed a JavaScript HTTP server task 55.8% faster when using GitHub Copilot [4]. 9 However, faster output does not necessarily imply secure output. The same body of literature shows that AI-generated code may appear functionally plausible while still containing vulnerabilities, insecure

defaults, or weak implementation choices that are difficult to detect through casual review [5], [6], [7]. 10 11 12

These risks become more serious in microservice-based backend systems. Although microservices improve modularity, scalability, and deployment flexibility, they also expand the attack surface through service-to-service communication, API exposure, dependency complexity, and distributed identity management [8], [9], [10]. 13 14 15 For a Node.js microservices environment, this means that insecure authentication logic, unsafe validation, dependency misuse, or poor access-control design can quickly propagate across multiple services and interfaces. 16 17 18

1.2 Motivation

The motivation for this research arises from a practical contradiction in modern software development. Developers are encouraged to use AI tools because they increase productivity, yet the same tools can generate code that is insecure, insufficiently validated, or poorly aligned with architectural constraints [4], [5], [6]. 9 10 11

This contradiction is reinforced by recent security evaluations. Early work found that approximately 40% of Copilot-generated programs were vulnerable in tested scenarios [5]. 10 More recent studies show that even strong coding agents often pass functional tests more often than security tests, and that current models still neglect important security properties during both generation and repair [7], [11]. 12 19

The present study is motivated by the need to close this gap in a concrete engineering context. Rather than treating AI-generated code security as a general concern, the thesis focuses on Node.js microservices and proposes a framework that integrates secure prompting, automated validation, and development workflow controls into the code generation loop itself. 20 21 22

1.3 Statement of the Problem

Despite the rapid adoption of AI-assisted coding tools, there is still no comprehensive security-first framework that governs the safe use of AI coding in production-oriented Node.js microservices. Existing approaches typically address only one part of the problem, such as secure prompting, benchmark evaluation, static analysis, or cloud-native defense mechanisms, without integrating them into a unified development workflow [7], [9], [10], [11]. 12 14 15 19

As a result, AI-generated backend code containing vulnerabilities such as injection flaws, weak authentication, insecure input handling, and access-control weaknesses may enter real systems without systematic scrutiny. This problem is intensified by what the proposal identifies as a verification gap, where developers increasingly trust generated code but do not examine it with enough depth to ensure its safety before integration. 23 24 25

Accordingly, the problem addressed by this thesis is the absence of a security-integrated framework for vibe coding that can guide the generation, checking, and controlled adoption of AI-produced Node.js microservice code in a way that improves security without discarding the productivity benefits of AI assistance. 26 27

1.4 Objectives

The general objective of this study is to design and evaluate a **Security-Integrated Development Framework (SIDF)** for vibe coding that systematically mitigates security risks in AI-generated Node.js microservices. 28

The specific objectives are:

1. To identify and classify common security vulnerabilities and reliability issues in AI-generated Node.js backend code. 28
2. To design a framework that embeds security-aware prompting, automated validation, and compatibility with existing DevSecOps workflows. 21
3. To implement the framework through a prototype tool, **VibeGuard**, that intercepts prompts, invokes LLMs, and scans generated code before integration. 20 21
4. To evaluate the framework by comparing standard vibe coding and VibeGuard-assisted development using metrics such as vulnerability density, development time, and test pass rates. 29 30

1.5 Methods

This research will adopt a **Design Science Research** approach because the study centers on the design, implementation, and validation of a new software engineering artefact. The main artefact

is the proposed Security-Integrated Development Framework, implemented through the VibeGuard prototype as a middleware layer for AI-assisted Node.js development. 29 20

The evaluation will use a controlled empirical study in which standard AI-assisted coding workflows are compared with framework-assisted workflows across representative backend tasks. The main evaluation measures are vulnerability density, development time, and test pass rate, supported by expert review and structured analysis of generated outputs. 29 31 32

1.6 Scope and Limitations

The study is limited to **Node.js microservices** and focuses on backend tasks such as authentication, CRUD APIs, validation middleware, and secure file handling. It considers security issues relevant to this context, especially OWASP Top 10-class vulnerabilities, input validation weaknesses, authentication flaws, and data-exposure risks. 30 33 34

The proposed solution is evaluated as a research prototype rather than a production-grade industrial platform. Its results are also bounded by the behavior of the selected LLMs, the limitations of integrated static analysis tools, the available participant sample, and the time and resource constraints of the study. 32 35 36 37

1.7 Application of Results

The expected outcome of this thesis is a practical framework and prototype that can help software teams adopt AI-assisted development more safely in backend microservice projects. In practice, the study is expected to provide a repeatable approach for combining rapid code generation with security checks, thereby reducing the likelihood that vulnerable AI-generated code will be integrated into production systems. 38

Academically, the study contributes to the growing body of research on AI-assisted software engineering by formalising the security dimension of vibe coding and by proposing an empirical evaluation strategy that connects code generation, validation, and software engineering workflow integration. 39

1.8 Organization of the Rest of the Thesis

The remainder of this thesis is organised as follows. **Chapter 2** reviews the literature on AI-assisted software engineering, security of AI-generated code, and secure microservice architecture.

Chapter 3 presents the related works and comparative analysis of existing approaches relevant to AI-assisted development and secure coding frameworks. **Chapter 4** describes the proposed framework and its architecture. **Chapter 5** discusses the implementation, experimentation, and evaluation results of the framework. Finally, **Chapter 6** concludes the thesis, summarises the findings, and outlines future work.

References

- 1, 2, 4, 5. [Missing authors]. (n.d.). Department of Computer Science - Brief History.
- 3, 40. Hailu, Fantahun Bishaw. (n.d.). Design and Evaluation of a Security-Integrated Vibe Coding Framework for Node.js Microservices.
6. Wong, Man-Fai et al.. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review. *Entropy*, 25(6), 888. <https://doi.org/10.3390/e25060888>
7. 10.48550/arxiv.2107.03374
8. 10.48550/arxiv.2510.12399
9. 10.48550/arxiv.2302.06590
10. 10.48550/arxiv.2108.09293
11. Negri-Ribalta, Claudia et al.. (2024). A systematic literature review on the impact of AI models on the security of code generation. *Frontiers in Big Data*, 7. <https://doi.org/10.3389/fdata.2024.1386720>
12. 10.48550/arxiv.2407.02395
13. 10.48550/arxiv.2408.03960
14. Rahaman, Md Shahidur et al.. (2023). Static-Analysis-Based Solutions to Security Challenges in Cloud-Native Systems: Systematic Mapping Study. *Sensors*, 23(4), 1755. <https://doi.org/10.3390/s23041755>
15. Rahaman, Md Shahidur et al.. (2023). Access Control Design Practice and Solutions in Cloud-Native Architecture: A Systematic Mapping Study. *Sensors*, 23(7), 3413. <https://doi.org/10.3390/s23073413>
16. Jayalath, Raveen Kanishka et al.. (2024). Microservice Vulnerability Analysis: A Literature Review with Empirical Insights. <http://arxiv.org/abs/2408.03960v1>
17. Rahaman, Md Shahidur et al.. (2023). Static-Analysis-Based Solutions to Security Challenges in Cloud-Native Systems: Systematic Mapping Study. *Sensors*, 23(4), 1755. <https://doi.org/10.3390/s23041755>
18. Rahaman, Md Shahidur et al.. (2023). Access Control Design Practice and Solutions in Cloud-Native Architecture: A Systematic Mapping Study. <https://doi.org/10.3390/s23073413>
19. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks

20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39. Hailu, Fantahun Bishaw. (n.d.). Design and Evaluation of a Security-Integrated Vibe Coding Framework for Node.js Microservices..

2. Literature Review

The literature review content will be here

Below is a **paste-ready Chapter 2 draft** written as a true **Literature Review** chapter, not a paper-by-paper Related Work chapter. It gives conceptual and technical background for the study area, the security problem, the evaluation landscape, and the engineering context in which your framework will operate. Citation numbering is kept consistent with the Chapter 1 sequence you already established, and new sources continue from that order.

Chapter 2

Literature Review

This chapter provides the technical and theoretical background for the study. It introduces the main concepts underlying AI-assisted software engineering, large language model based code generation, secure coding, code quality evaluation, and microservice security. It also outlines the main engineering ideas and tools that inform the proposed framework. Unlike the related work chapter, the focus here is not on individual papers as isolated studies, but on the broader concepts, methods, and technical foundations that define the research domain.

2.1 AI-Assisted Software Engineering

AI-assisted software engineering refers to the use of machine learning models, especially large language models, to support software development tasks such as code generation, completion,

repair, summarization, translation, defect detection, and documentation [1], [2]. 1 2 In this setting, source code is treated as a structured language that can be modelled through token prediction, contextual inference, and sequence generation. This is possible because programming languages exhibit regular syntax, strong structural constraints, and recurring patterns that are learnable from large corpora of public code [1], [6]. 3 4

The practical value of AI-assisted programming lies in its ability to reduce effort in repetitive or pattern-heavy tasks. Research in this area reports use cases ranging from natural-language-to-code generation to automated program improvement, code explanation, and repository navigation [1], [12], [13], [14]. 5 6 7 8 The field has therefore moved beyond simple line completion and now includes tool-using agents capable of editing files, running tests, querying repositories, and iterating on generated code in more realistic software engineering environments [12], [13], [14]. 6 9 10

This evolution is important for the present thesis because the target problem is not limited to isolated code snippets. The study is concerned with backend microservice development, where code generation takes place within repositories, service boundaries, configuration files, and deployment workflows. In such settings, the quality of generated code depends not only on syntax or local correctness, but also on architecture, interface contracts, dependencies, security constraints, and operational assumptions.

2.2 Large Language Models for Code Generation

Large language models for code generation are typically transformer-based systems trained on large corpora of source code and related textual artefacts. They accept prompts in the form of comments, partial code, specifications, or natural language requirements, and then generate code token by token until a completion condition is reached [2], [16]. 2 11

From a software engineering perspective, this capability changes the role of the developer. Instead of writing every implementation detail manually, the developer increasingly acts as a task specifier, reviewer, and orchestrator of generated artefacts. The literature review on vibe coding describes this shift as a move from assistance to more autonomous development workflows, where developers provide high-level intent and assess system outputs mainly through behaviour and

outcomes [3]. 12 The same review also notes that effective use of these systems depends on prompt engineering, context management, and structured interaction rather than unrestricted prompting alone [3]. 13

This transition has led to the rise of agent-based coding systems. Repository-level tools such as SWE-agent, AutoCodeRover, and CodeAgent demonstrate that realistic software engineering requires more than next-token prediction. It requires file navigation, code search, structural understanding, test execution, debugging, and context-aware modification of existing codebases [12], [13], [14]. 6 9 10 14 This background is central to the present study because the proposed framework is designed for AI-assisted Node.js microservice development, not for standalone algorithmic exercises.

2.3 Vibe Coding as a Development Context

Vibe coding can be understood as a high-level, conversational style of software production in which AI systems generate substantial portions of implementation from natural language intent [3]. 12 It represents a stronger form of AI assistance than autocomplete because the model is expected to infer design decisions, implementation structures, and sometimes even project-level changes from informal instructions.

The attraction of vibe coding lies in rapid prototyping and reduced manual effort. However, the same abstraction that makes it productive also weakens direct human inspection of generated logic. As a result, developers may accept implementations that appear correct at the surface level but still contain architectural inconsistencies, hidden defects, or security vulnerabilities [3], [11]. 15 16 17

This issue becomes more visible in repository-level evaluation. Recent benchmark research shows that many earlier secure-code benchmarks focus on single files or single functions, while practical vibe coding often involves multi-file editing, repeated turns, environment feedback, and large repository context [11]. 18 19 The distinction matters because a secure function in isolation does not guarantee secure behaviour once dependencies, API gateways, configuration files, authentication flows, and data-handling paths are considered together.

2.4 Code Quality, Correctness, and Evaluation of AI-Generated Code

Code generation quality cannot be reduced to syntactic validity alone. In software engineering, code quality usually spans validity, correctness, reliability, maintainability, and security [17]. 20 Accordingly, the evaluation of AI-generated code requires broader criteria than whether a model can compile or produce a plausible-looking answer.

A major challenge in this area is benchmark adequacy. Traditional evaluation benchmarks such as HumanEval and MBPP are useful for measuring function-level correctness, but they capture only one part of software engineering capability [15], [18]. 21 22 HumanEval+ extends functional testing by greatly increasing the number and diversity of test cases, showing that earlier pass-rate estimates may overstate model correctness when test coverage is weak [18]. 23 LiveCodeBench further broadens evaluation by covering self-repair, code execution, and test output prediction in addition to code generation, thereby better reflecting practical development activities [15]. 24 25

Repository-level evaluation adds another layer. Real software engineering requires the ability to understand existing structures, modify related files consistently, and operate within large codebases. Benchmarks such as SWE-bench, CodeAgentBench, and newer repository-oriented security benchmarks move closer to this reality by testing whether an AI system can solve engineering problems inside actual repositories [13], [14], [15]. 10 22 26

For the present thesis, this literature has two implications. First, evaluation should include more than raw task completion. Second, backend code generated for microservices should be assessed through both functionality and security-oriented measures, since a functionally correct service may still be unsafe in production.

2.5 Security of AI-Generated Code

The security of AI-generated code is now a well-established concern in the literature. The systematic review in this area concludes that AI-generated code is not necessarily secure and that recurring vulnerabilities are commonly observed across different studies, languages, and tools [6]. 27 28 The same review notes that common weakness classes from the MITRE CWE Top 25 appear frequently in generated outputs and that dedicated security processes are still necessary when AI participates in software production [6]. 29 30 31

Empirical studies reinforce this view. Security evaluations of Copilot-generated code found vulnerable outputs in a substantial share of tested programs [5]. 32 Broader code-security evaluations show that current models often neglect security in both generation and repair tasks, even when they perform strongly on functional code production [7]. 33 34 Recent literature also shows that security problems are not limited to general bugs. They include misuse of security-sensitive APIs, poor authentication logic, unsanitised input handling, hardcoded secrets, and weak configuration decisions [6], [7]. 35 36

A further complication is that insecure code may still look correct to both the model user and downstream reviewers. The literature review highlights that human trust in AI-generated code can be unjustifiably high, especially when the output is readable, idiomatic, and apparently complete [6]. 30 This is particularly relevant in backend service development, where the most damaging vulnerabilities often hide behind normal application behaviour until triggered by malicious input or complex interaction paths.

2.6 Security Hardening, Prompting, and Automated Validation

The literature shows that AI-generated code security can be improved, but current solutions are fragmented. One line of work focuses on model-side hardening. Controlled code generation methods such as SVEN attempt to steer models toward secure outputs while preserving functional correctness [16]. 37 Another line of work emphasises prompt quality, contextual guidance, and security-aware prompting, arguing that carefully structured prompts can influence whether vulnerabilities are introduced or avoided [6]. 38 31

A third line of work focuses on evaluation and validation. CodeSecEval provides a benchmark containing 180 samples across 44 vulnerability types to evaluate secure code generation and secure repair more systematically [7]. 34 The literature review on AI-generated code security also stresses that testing, static analysis, fuzzing, and policy-oriented verification remain necessary because generated code can contain non-obvious flaws that are not reliably prevented by generation alone [6]. 39

This background is directly relevant to the proposed thesis framework. The planned system does not assume that prompting alone is enough, nor that post-generation scanning alone is enough.

Instead, the relevant literature suggests that secure AI-assisted development benefits from combining prompt discipline, automated validation, and explicit software process controls. That combination motivates the security-integrated orientation of the proposed framework.

2.7 Microservices, Cloud-Native Security, and Access Control

Microservice architecture decomposes a system into small, independently deployable services that communicate through APIs, messaging, or other lightweight mechanisms. This architecture improves maintainability, scaling flexibility, and modular deployment, but it also introduces additional security complexity [8]. 40 41 Compared with monolithic systems, microservices create more communication boundaries, more configuration surfaces, more dependencies, and more identity and trust relationships that must be secured.

Recent review work identifies 126 distinct vulnerabilities across 62 studies of microservice architectures and validates them through scans of multiple benchmark applications [8]. 40 42 43 The same study shows that different scanning tools detect different weaknesses, which implies that single-tool assessment is often insufficient in complex service environments [8]. 44

Cloud-native security literature complements this view by focusing on defensive mechanisms. Static-analysis-based studies show that microservice security requires attention to API protection, service identity, confidential data handling, least privilege, and the relationship between attacks and defensive controls [9]. 45 46 47 Access-control studies further identify authentication and authorization as central concerns in cloud-native systems and show the practical importance of OAuth2, JWT, mutual TLS, and architectural-level security design patterns [10]. 48 49 50

These ideas are closely related to zero-trust thinking, even when that term is not always used explicitly in every study. In practical terms, the recurring design principles are explicit verification, strong service identity, limited privilege, protected APIs, and careful control of service-to-service communication [9], [10]. 46 50 For a security-integrated AI coding framework, these principles imply that generated code should be evaluated not only for syntax and functionality, but also for how it handles access control, identity, trust boundaries, and API exposure.

2.8 Node.js Microservices and Development Tooling Context

The present thesis applies these concepts in the context of Node.js backend microservices. The target environment consists of JavaScript-based backend services using RESTful APIs, message queues, and SQL databases, with security emphasis on injection flaws, authentication problems, and data exposure. The intended workflow also assumes integration with GitHub, Docker, and standard CI/CD tooling, together with automated static analysis and testing. 51 52

This environment is important for two reasons. First, Node.js is widely used for lightweight service-oriented backend development, which makes it a suitable setting for studying practical AI-assisted coding. Second, microservices amplify the consequences of insecure generated code because a weak service can become an entry point into a broader system. In such an environment, validation should occur as early as possible, preferably during the generation workflow rather than only after code has already been integrated. 53 54 55

2.9 Chapter Summary

This chapter presented the background literature necessary for understanding the research domain. It discussed AI-assisted software engineering, the role of large language models in code generation, the emergence of vibe coding, and the need for broader evaluation criteria that include correctness, reliability, and repository-level capability [1], [2], [3], [12], [13], [14], [15], [18]. 1 2 12 56 7 8 57 58

It also established that AI-generated code security is a persistent concern and that secure AI-assisted development requires more than model capability alone. Prompt quality, automated validation, testing, static analysis, access control, and cloud-native security principles all remain important [5], [6], [7], [8], [9], [10], [16]. 32 59 60 61 62 63 64

For the present thesis, the main implication is that secure vibe coding for Node.js microservices must be treated as an engineering problem that spans generation, validation, and service architecture rather than code generation alone. The next chapter therefore examines related work more directly and positions the proposed framework against existing approaches in the literature. 53 54 55

Citation continuation for Chapter 2

If you keep the Chapter 1 numbering unchanged, the new references introduced here continue as:

- [12] AutoCodeRover: Autonomous Program Improvement. 56
- [13] SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 7
- [14] CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. 8
- [15] LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. 57
- [16] Large Language Models for Code: Security Hardening and Adversarial Testing. 64
- [17] Evaluating the Code Quality of AI-Assisted Code Generation Tools. 65
- [18] Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. 58

References

1. Wong, Man-Fai et al.. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review. Entropy, 25(6), 888. <https://doi.org/10.3390/e25060888>
2. 10.48550/arxiv.2107.03374
- 3, 5. Wong, Man-Fai et al.. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review. Entropy, 25(6), 888. <https://doi.org/10.3390/e25060888>
- 4, 27, 28, 29, 30, 31, 36, 38, 39. Negri-Ribalta, Claudia et al.. (2024). A systematic literature review on the impact of AI models on the security of code generation. Frontiers in Big Data, 7. <https://doi.org/10.3389/fdata.2024.1386720>
3. Zhang, Yuntong et al.. (2024). AutoCodeRover: Autonomous Program Improvement. Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 1592-1604. <https://doi.org/10.1145/3650212.3680384>
4. 10.48550/arxiv.2405.15793
5. Zhang, Kechi et al.. (2024). CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 13643-13658. <https://doi.org/10.18653/v1/2024.acl-long.737>
6. Yang, John et al.. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. <http://arxiv.org/abs/2405.15793v3>
- 10, 14. Zhang, Kechi et al.. (2024). CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 13643-13658. <https://doi.org/10.18653/v1/2024.acl-long.737>
- 11, 37. He, Jingxuan and Vechev, Martin. (2023). Large Language Models for Code: Security Hardening and Adversarial Testing. Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 1865-1879. <https://doi.org/10.1145/3576915.3623175>

7. 10.48550/arxiv.2510.12399
13, 15, 16. Ge, Yuyao et al.. (2025). A Survey of Vibe Coding with Large Language Models. <http://arxiv.org/abs/2510.12399v2>
8. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks
18, 19, 26. Zhao, Songwen et al.. (2025). Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. <http://arxiv.org/abs/2512.03262v2>
9. Yetiştiren, Burak et al.. (n.d.). Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. <http://arxiv.org/abs/2304.10778v2>
21, 23. Liu, Jiawei et al.. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. <http://arxiv.org/abs/2305.01210v3>
22, 24, 25. Jain, Naman et al.. (2024). LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. <http://arxiv.org/abs/2403.07974v2>
10. 10.48550/arxiv.2108.09293
33, 34, 35. Wang, Jiexin et al.. (2024). Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval. <http://arxiv.org/abs/2407.02395v2>
40, 41, 42, 43, 44. Jayalath, Raveen Kanishka et al.. (2024). Microservice Vulnerability Analysis: A Literature Review with Empirical Insights. <http://arxiv.org/abs/2408.03960v1>
45, 46, 47. Rahaman, Md Shahidur et al.. (2023). Static-Analysis-Based Solutions to Security Challenges in Cloud-Native Systems: Systematic Mapping Study. Sensors, 23(4), 1755. <https://doi.org/10.3390/s23041755>
48, 49, 50. Rahaman, Md Shahidur et al.. (2023). Access Control Design Practice and Solutions in Cloud-Native Architecture: A Systematic Mapping Study. <https://doi.org/10.3390/s23073413>
51, 52, 53, 54, 55. Hailu, Fantahun Bishaw. (n.d.). Design and Evaluation of a Security-Integrated Vibe Coding Framework for Node.js Microservices..

11. Zhang, Yuntong et al.. (2024). AutoCodeRover: Autonomous Program Improvement. Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 1592-1604. <https://doi.org/10.1145/3650212.3680384>
12. 10.48550/arxiv.2403.07974
13. 10.48550/arxiv.2305.01210
14. Negri-Ribalta, Claudia et al.. (2024). A systematic literature review on the impact of AI models on the security of code generation. Frontiers in Big Data, 7. <https://doi.org/10.3389/fdata.2024.1386720>
15. 10.48550/arxiv.2407.02395
16. 10.48550/arxiv.2408.03960
17. Rahaman, Md Shahidur et al.. (2023). Static-Analysis-Based Solutions to Security Challenges in Cloud-Native Systems: Systematic Mapping Study. Sensors, 23(4), 1755. <https://doi.org/10.3390/s23041755>
18. Rahaman, Md Shahidur et al.. (2023). Access Control Design Practice and Solutions in Cloud-Native Architecture: A Systematic Mapping Study. Sensors, 23(7), 3413. <https://doi.org/10.3390/s23073413>
19. He, Jingxuan and Vechev, Martin. (2023). Large Language Models for Code: Security Hardening and Adversarial Testing. Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 1865-1879. <https://doi.org/10.1145/3576915.3623175>
20. 10.48550/arxiv.2304.10778

3. Related Work

The Related work content will be here.

Chapter 3. Related Work

This chapter reviews prior studies that are closely related to the design and evaluation of a security-integrated vibe coding framework for Node.js microservices. The reviewed works are organized into three themes: security assessment of AI-generated code, security-aware generation and vulnerability detection, and agentic repository-level software engineering.

3.1 Security assessment of AI-generated code

One of the earliest and most influential studies in this area is the work by Pearce et al., which systematically assessed the security of GitHub Copilot’s code suggestions. The authors evaluated Copilot across 89 security-relevant scenarios and generated 1,689 programs, organizing the study around diversity of weakness, diversity of prompt, and diversity of domain.⁴⁵⁶ To identify insecure outputs, they combined CodeQL analysis with manual inspection and reported that approximately 40% of the generated programs were vulnerable.⁴⁵

The main strength of this paper is its disciplined experimental structure, which showed that security outcomes are sensitive not only to the target weakness but also to small prompt variations and domain changes.⁶⁷⁸ For example, in the SQL injection prompt-diversity experiment, the authors evaluated 17 prompt variants and found 152 vulnerable programs out of 407 valid outputs, showing that even subtle phrasing changes can shift the security profile of the generated code.⁷⁹ However, the study also has clear limitations. The authors themselves note that the scenarios were artificial and did not capture the full messiness of real-world codebases and wider project context.¹⁰ The key lesson from this work is that AI code assistants should not be treated as security-neutral productivity tools; they must be paired with security-aware validation and careful human review.¹¹

A more recent evaluation framework is CodeSecEval, introduced to assess secure code generation and insecure code repair more rigorously. This study created a curated Python benchmark with 180 samples across 44 vulnerability types, and importantly, the dataset includes executable secure and insecure code together with test cases, enabling automated Pass@k evaluation.¹²¹³¹⁴ The authors used this benchmark to evaluate seven modern code models and showed that direct secure generation remains weak across many vulnerability classes. They also demonstrated that

vulnerability-aware problem descriptions can substantially improve security performance, for example raising Claude 3 Opus from Pass@1 13.83 to 39.89 and Pass@5 15.96 to 49.55 on CodeSecEval.¹⁵

The strength of CodeSecEval lies in its executable benchmark design, which overcomes important limitations of earlier datasets that lacked secure reference code, complete runnable samples, or automated evaluation support.¹⁶¹⁷ At the same time, the study shows that gains are uneven across vulnerability categories, with some classes still scoring 0.0 under direct generation for GPT-4, and the authors explicitly note that generalizability to other programming languages remains an open issue.¹⁵¹⁸ The lesson for the present thesis is that evaluation of vibe coding systems must go beyond functional correctness and include executable, vulnerability-specific security testing.

3.2 Security-aware generation and vulnerability detection

He and Vechev approached the problem from a different angle by asking whether code models can be actively steered toward secure or insecure generation. They formulated a new task called controlled code generation and proposed SVEN, a method that uses property-specific continuous prefixes to guide a frozen code language model without changing its weights.¹⁹²⁰²¹ For training, they curated vulnerability data from CrossVul, Big-Vul, and VUDENC, and for evaluation they used scenario-based security testing together with HumanEval for functional correctness.²²²³²⁴ Their results are strong: for CodeGen-2.7B, the secure-program rate increased from 59.1% to 92.3% under SVEN-based hardening, while functional correctness remained close to the original model on HumanEval.²⁵²⁶²⁷

The main strength of SVEN is that it shows security can be incorporated before code is emitted, rather than relying only on post-generation scanning or repair.¹⁹²⁸ This is important because it reframes security as a controllable generation property rather than merely a downstream detection problem. Still, the study remains constrained by its evaluation setup. Its core results are centered on scenario-based testing and mainly on the CodeGen family, and when the authors extended the method to SantaCoder they had to exclude several C/C++ CWEs because the model was not sufficiently trained for those languages.²³²⁹ The lesson here is that security-aware generation is

promising, but practical deployment requires attention to cross-model and cross-language generalization.

A complementary line of work is represented by IRIS, which combines large language models with static analysis for vulnerability detection. IRIS first extracts candidate APIs from a Java project, uses an LLM to label sources and sinks for a target vulnerability class, converts these labels into CodeQL-style specifications, runs taint analysis, and then uses an LLM again to filter false positives.³⁰³¹ The system was evaluated on CWE-Bench-Java, a dataset of 120 vulnerabilities across large compilable Java projects averaging about 300,000 lines of code each.³² With GPT-4, IRIS detected 55 vulnerabilities, which is 28 more than CodeQL, achieved a lower average false discovery rate than CodeQL, and identified four previously unknown vulnerabilities in later project versions.³³³⁴³⁵

The major strength of IRIS is that it shows how symbolic analysis and LLM reasoning can complement each other. CodeQL supplies precise path-based analysis, while the LLM helps infer project-specific security knowledge that static analyzers often miss.³⁶³¹ However, the paper also reports important limitations: IRIS still misses many vulnerabilities, it incurs additional analysis cost because it makes many LLM calls, its generalization beyond Java is unknown, and the resulting reports still do not fully match what developers need in practice.³⁷ The lesson for this thesis is that secure vibe coding frameworks should not rely on a single validation technique. Instead, they should combine static analysis, contextual reasoning, and developer-usable feedback.

3.3 Agentic repository-level software engineering

Beyond single-function generation, recent work has started to address repository-level coding with tools and agents. CodeAgent is one of the clearest examples. The paper presents a tool-integrated agent framework for repo-level code generation, where an LLM can use external tools for information retrieval, code implementation, and testing, supported by strategies such as ReAct, OpenAIFunc, rule-based control, and tool planning.³⁸³⁹⁴⁰ To evaluate the approach, the authors constructed CodeAgentBench, a repository-level benchmark with 101 functions and classes from real GitHub projects, selected across five common software topics.⁴¹⁴² Their experiments on nine LLMs showed repo-level pass-rate improvements ranging from 2.0 to 15.8 points, with the GPT-

4 setting achieving a 72.7% relative improvement over the non-agent baseline.⁴³⁴⁴ They also reported gains on HumanEval, where Pass@1 improvements ranged from 6.1 to 9.7 across selected models.⁴⁵

The main strength of CodeAgent is that it treats coding as a tool-mediated repository task rather than a pure next-token prediction problem.⁴⁶⁴⁰ This makes the method much closer to practical software development, where navigation, dependency inspection, and testing are essential. However, the study is explicitly an early exploration. The authors acknowledge that contamination risk in the benchmark is not yet fully ruled out, that only simple tools were incorporated, and that comparisons with commercial IDE products were manual rather than rigorously standardized.⁴⁷ The lesson for this thesis is that agentic support is useful for handling complex repositories, but security assurance must be added as a first-class concern rather than assumed to follow from better task completion.

AutoCodeRover moves even closer to practical maintenance by focusing on autonomous program improvement for real GitHub issues. The method combines LLM reasoning with abstract syntax tree based code search, iterative retrieval, and optional test-based fault localization, while using only the natural-language issue description and the repository checkout as required inputs.⁴⁸⁴⁹⁵⁰ On SWE-bench lite, AutoCodeRover resolved 19% of issues at pass@1 and 26% at pass@3, averaging 195 seconds and about \$0.43 at pass@1, and it compared favorably with Swe-agent in both effectiveness and cost efficiency on that benchmark.⁵¹⁵²⁵³

A major strength of AutoCodeRover is its software-engineering orientation. Instead of treating a repository as a flat bag of files, it exploits program structure and fine-grained retrieval, which helped it uniquely solve cases where AST-level context mattered.⁵⁴⁵⁵ Its weaknesses are equally informative. The authors show that many failures came from unimplemented search APIs and, more fundamentally, from the difficulty of inferring developer intent or specifications from issue descriptions alone.⁵⁵⁵⁶⁵⁷ The lesson is that autonomous coding agents benefit greatly from structured retrieval and validation loops, but they still require stronger specification support and cannot be assumed to produce secure outputs automatically.⁵⁷⁵⁰

3.4 Summary of gaps in the reviewed works

The reviewed studies collectively establish three major points. First, AI-assisted code generation is clearly useful, but it can still produce insecure outputs at non-trivial rates, as shown by the approximately 40% vulnerability rate reported in the Copilot study and the weak direct secure-generation performance observed across many CodeSecEval tasks.⁴¹⁵ Second, security can be improved either before generation, as in SVEN, or after generation, as in IRIS, but these solutions are typically evaluated on specific languages or narrowly defined tasks such as Python security prompts, Java vulnerability detection, or benchmark scenarios rather than full end-to-end development workflows.¹⁴³²²²³⁷ Third, agentic repository-level systems such as CodeAgent and AutoCodeRover improve coding-task completion, but their primary emphasis is functional success, retrieval efficiency, and issue resolution, not security-integrated development for backend microservices.⁴³⁵¹⁵²

This leaves a clear research gap for the present thesis. Existing works do not provide a security-integrated framework tailored to AI-augmented Node.js backend microservices, and they do not combine security-aware prompting, automated validation, and workflow-level integration into a single practical vibe coding pipeline.⁵⁸⁵⁹⁶⁰ The proposed research addresses this gap by designing the Security-Integrated Development Framework and implementing VibeGuard as a Node.js middleware tool that intercepts prompts, invokes LLMs, scans generated code for vulnerabilities, and supports empirical comparison between standard vibe coding and a security-integrated workflow on Node.js microservice tasks.⁵⁹⁶¹⁶²⁶³

References

- 1, 2, 3. [Missing authors]. (n.d.). Department of Computer Science - Brief History.
- 4, 5, 6, 7, 8, 9, 10, 11. Pearce, Hammond et al.. (2021). Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. <http://arxiv.org/abs/2108.09293v3>
- 12, 13, 14, 15, 16, 17, 18. Wang, Jiexin et al.. (2024). Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval. <http://arxiv.org/abs/2407.02395v2>
- 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29. He, Jingxuan and Vechev, Martin. (2023). Large Language Models for Code: Security Hardening and Adversarial Testing. Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 1865-1879. <https://doi.org/10.1145/3576915.3623175>
- 30, 31, 32, 33, 34, 35, 36, 37. Li, Ziyang, Dutta, Saikat, and Naik, Mayur. (2024). IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. <http://arxiv.org/abs/2405.17238v3>
- 38, 39, 40, 41, 42, 43, 44, 45, 46, 47. Zhang, Kechi et al.. (2024). CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 13643-13658. <https://doi.org/10.18653/v1/2024.acl-long.737>
- 48, 49, 50, 51, 52, 53, 54, 55, 56, 57. Zhang, Yuntong et al.. (2024). AutoCodeRover: Autonomous Program Improvement. Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 1592-1604. <https://doi.org/10.1145/3650212.3680384>
- 58, 59, 60, 61, 62, 63. Hailu, Fantahun Bishaw. (n.d.). Design and Evaluation of a Security-Integrated Vibe Coding Framework for Node.js Microservices..

4. The Proposed Solution

PROPOSED SOLUTION CONTENT WILL BE HERE.

Please note that the titles and number of chapters from this point onwards may vary from thesis to thesis. Hence, this section should be considered as a general guideline and should be tailored towards the nature of each thesis.

Below is a full polished **Chapter 4** in thesis prose, structured and ready to paste into your document. The wording keeps the solution technical, research-oriented, and aligned with the thesis title, proposal, and university guideline. The chapter is written as a **proposed framework chapter**, not as an implementation-results chapter, which is consistent with the guideline's expectation that this chapter should define the system architecture, structure, behavior, interactions, and how the solution works.¹²³

4.1 Chapter Overview

This chapter presents the proposed solution of the study, namely the **Security-Integrated Development Framework (SIDF)** and its prototype instantiation, **VibeGuard**, for secure AI-assisted development of Node.js microservices.⁴⁵ In accordance with the university guideline, the chapter focuses on the conceptual structure and behavior of the solution by explaining its architecture, components, data and control flow, and the mechanisms through which it addresses the research problem.¹²³

The chapter begins by identifying the design goals and requirements that guided the proposed framework. It then presents the overall architecture of SIDF, explains the major components of VibeGuard, describes the operational workflow of secure vibe coding, and discusses the security integration mechanisms required for Node.js microservices. Finally, the chapter explains how the proposed solution addresses the identified research problem and clarifies its scientific contribution and practical value.⁶⁷⁸⁹

4.2 Design Goals and Solution Requirements

The design of the proposed solution is motivated by a central concern established in the proposal: AI-assisted vibe coding increases development speed, but this speed is not matched by an equivalent increase in developers' ability to verify correctness and security in generated code.¹⁰ This imbalance creates what the proposal calls a **verification paradox**, where developers gain productivity through generation yet lose assurance because they often accept AI output without sufficiently rigorous review, especially in security-sensitive software contexts.¹⁰¹¹

This problem is particularly significant for Node.js backend microservices. The proposal identifies this domain as practically important and under-served by existing secure AI-assisted

development frameworks, despite the fact that it commonly involves security-critical features such as authentication, CRUD APIs, validation middleware, secure file handling, message queues, and SQL-backed services.¹²¹³¹⁴ It further notes that current LLM-generated code frequently reproduces insecure patterns such as unparameterized queries, hardcoded secrets, weak authentication logic, unsanitized input handling, and insecure serialization paths.¹⁵

The proposed framework is therefore designed around six main goals. First, it must support **security by design**, so that security is introduced at the point of code generation rather than only after the code has already entered the development workflow.⁵¹⁶ Second, it must provide **generation-time validation**, so that AI-generated code is evaluated immediately after generation and before repository integration.⁴¹⁷ Third, it must remain **compatible with practical development workflows**, particularly Git-based version control, Dockerized services, and standard CI/CD pipelines.¹⁸⁸

Fourth, the framework must be **microservice-aware**, since the solution is not intended for generic programming tasks alone but specifically for Node.js microservice systems with distributed interfaces and service boundaries.¹⁴¹⁹ Fifth, it must preserve **human-centered governance**, meaning that the framework assists and constrains developers but does not remove their final responsibility for accepting or rejecting code.²⁰ Sixth, it must support **auditability and traceability** through systematic provenance logging of prompts, generated outputs, validation findings, and decisions, since current AI-assisted workflows lack standardized trust and audit mechanisms.²¹¹¹

From these goals, the main solution requirements can be stated as follows. The framework must intercept developer prompts before they reach the LLM, classify the request according to its risk profile, enrich security-sensitive prompts with appropriate constraints, orchestrate code generation, validate outputs using static analysis, apply policy-driven acceptance or rejection logic, log the full generation trace, and remain usable within ordinary Node.js microservice development workflows.²²⁵¹⁸²³

4.3 Overview of the Proposed Solution

The proposed solution of this thesis is the **Security-Integrated Development Framework (SIDF)**, a framework for governing AI-assisted code generation in a way that embeds security controls directly into code generation workflows.⁴⁷ SIDF is instantiated through a prototype called **VibeGuard**, which is designed as a Node.js-based middleware and command-line tool that sits between developer prompts, large language model APIs, and security analysis tools.²²⁵

Conceptually, SIDF treats code generation as a controlled software engineering process rather than as unconstrained conversational prompting. In a conventional workflow, a developer submits a natural-language request directly to an LLM, receives generated code, and then decides informally whether the output is acceptable. In the proposed framework, this direct path is replaced by a mediated pipeline in which prompts are intercepted, risk-assessed, enriched where necessary, submitted to the model under explicit security constraints, and then validated before they can be accepted as usable output.⁴²²⁵²⁴

The framework is designed to support common Node.js microservice development tasks such as implementing authentication flows, CRUD services, validation middleware, secure API integration, and secure file handling.¹³¹⁴ In each case, SIDF introduces a control loop that attempts to reduce the probability that insecure patterns pass from a prompt into generated code and then into the repository or deployment pipeline. The framework therefore combines preventive controls, detective controls, and governance mechanisms into a single structured workflow.⁵²¹²⁵

From a design science perspective, SIDF is the conceptual research artifact, while VibeGuard is the executable prototype used to instantiate and evaluate that artifact in practice.⁴⁹ This distinction is important because the research contribution is not limited to building a tool. The thesis also contributes a formal framework that defines how secure vibe coding should be structured, governed, and evaluated in a production-relevant software engineering setting.⁹²⁶

4.4 Architecture of the Security-Integrated Development Framework

The overall architecture of SIDF is organized into six cooperating layers: the **developer interaction layer**, the **prompt intelligence layer**, the **LLM orchestration layer**, the **validation and policy layer**, the **provenance and feedback layer**, and the **delivery and integration layer**. This layered organization is appropriate because the university guideline expects the proposed solution chapter to define the components of the system, their interactions, and the flow of information among them.¹²²⁷

At the highest level, the architecture begins with developer intent and ends with a gated decision on whether generated code should be accepted, revised, or rejected. Between these endpoints, the framework creates a controlled path through which code generation is transformed from a raw conversational act into a secure engineering process. This architectural decision directly addresses the lack of structured methodology for integrating vibe coding into rigorous development lifecycles, a gap explicitly identified in the proposal.¹¹²¹

4.4.1 Developer Interaction Layer

The developer interaction layer is the entry point of the framework. Its purpose is to capture a development task expressed in natural language and ensure that the request is routed through the framework rather than sent directly to an LLM. In the proposed prototype, this layer is implemented through the VibeGuard command-line interface and middleware wrapper described in the proposal.²²⁵

This layer is intentionally minimal. It does not attempt to replace the developer's integrated development environment or general-purpose coding assistant. Instead, it establishes a controlled access point for AI-assisted generation, ensuring that each coding request enters the same governed pipeline. This decision is especially important in vibe coding contexts because developers often interact with AI systems in an informal and rapid manner, which increases the risk of bypassing structured verification.¹⁰²⁸

4.4.2 Prompt Intelligence Layer

The prompt intelligence layer performs the first core reasoning task of the framework. It inspects the developer's request, identifies whether the task is security-sensitive, and determines whether prompt enrichment is required. The proposal explicitly states that the framework will provide automated enrichment of prompts and security-aware prompt engineering support.¹⁸²⁹

The logic of this layer follows a straightforward principle. Not all prompts require the same level of control. A request to generate a simple data formatter is not equivalent to a request to implement login middleware, file upload handling, token generation, or a database-backed API endpoint. The framework therefore classifies prompts according to risk categories derived from the architectural and security context of the requested task. Prompts involving authentication, authorization, SQL queries, secrets, user input, file operations, inter-service communication, or external API access are treated as security-sensitive and are enriched with security constraints before model invocation.¹⁵¹⁸³⁰

The proposal's technology and security scope provides the basis for these enrichments. The framework focuses on OWASP Top 10 vulnerabilities, with emphasis on Node.js-specific risks such as injection, authentication flaws, and data exposure.¹⁸ Accordingly, the prompt intelligence layer may inject guidance such as parameterized database access, secure secret handling, input validation, secure file path management, token-based authentication expectations, least privilege assumptions, or API gateway awareness when such concerns are relevant to the task.³¹³²³³

4.4.3 LLM Orchestration Layer

The LLM orchestration layer manages communication with the underlying language model. It receives the enriched prompt, forwards it to the selected model provider, captures the response, and normalizes the generated output into a form suitable for downstream validation and logging.²²²¹ The proposal indicates that the research is intended to work with contemporary frontier models while remaining extensible across different model versions and providers.¹⁴

This layer is designed to isolate the rest of the framework from the volatility of LLM ecosystems. The proposal correctly notes that model behavior changes over time as new versions, parameter updates, and training adjustments are released, meaning that findings obtained on one model version cannot automatically be generalized to future versions.³⁴³⁵ By isolating model access behind a stable orchestration layer, SIDA can preserve a consistent security workflow even when the underlying generation engine changes.

A second responsibility of this layer is metadata capture. For each generation event, the system should record the model identifier, timestamp, enriched prompt, output artifact, and any generation settings relevant for later auditing or empirical analysis. This is necessary because one of the explicit objectives of the proposal is to enable structured comparison between baseline vibe coding and VibeGuard-assisted workflows using measurable outcomes such as vulnerability density, development time, and test pass rate.²³²¹

4.4.4 Validation and Policy Layer

The validation and policy layer is the security core of the framework. After code is generated, it is passed through an automated validation stage before it is accepted as an approved output. This stage implements the proposal's objective of embedding automated validation into the vibe coding workflow using static analysis tools and security-aware checks.⁵¹⁸³⁶

This layer contains four logical elements. The first is the **static analysis engine**, which runs SAST tools on the generated Node.js code. The proposal specifically identifies open-source tools such as ESLint with security plugins and Semgrep as part of the intended solution scope.³⁶ The second is the **rule and policy engine**, which checks whether the output violates framework-level constraints, such as mandatory authentication patterns, restricted secret handling, or unsafe API behavior. The third is the **risk classification unit**, which maps findings to severity categories. The fourth is the **decision gate**, which determines whether the code should be accepted, sent back for regeneration, or escalated to the developer for review.

The choice to place automated validation immediately after generation is strongly supported by the research base. Recent work on secure evaluation of LLM-generated code identifies static analysis as a preferred mechanism for early-stage detection of vulnerabilities in generated source code.³⁷³⁸ Secure vibe coding literature also argues that the prompt-generate-validate cycle must include real-time or near-real-time feedback so that insecure output can be flagged or corrected before it becomes part of the normal development flow.²⁴¹⁷

The decision gate is the element that converts analysis into action. This is a major difference between SIDF and a simple security scanner wrapper. A conventional scanner may only report findings and leave interpretation entirely to the developer. By contrast, SIDF interprets findings operationally. Critical injection paths, hardcoded secrets, authentication weaknesses, or unsafe file system patterns may result in automatic rejection and regeneration. Moderate-severity issues may lead to warnings and explicit developer confirmation. Clean or policy-compliant outputs can be approved and passed forward. In this way, the framework embeds security enforcement into the act of generation itself.⁵¹⁵³⁹

4.4.5 Provenance and Feedback Layer

The provenance and feedback layer provides the persistent memory of the framework. The proposal emphasizes that current AI-assisted workflows lack systematic provenance logging and that developers often lack trust metrics for AI-generated code.¹¹²¹ The proposed solution addresses this by recording the full trace of each coding session, including the original prompt, enriched prompt, model information, generated output, validation findings, decision outcome, and any subsequent regeneration events.²¹²³

This layer serves several important purposes. First, it supports **auditability**, which is essential when AI-generated code is used in security-sensitive applications. Second, it supports **empirical evaluation**, since the thesis aims to compare vulnerability density, development time, and test outcomes between standard vibe coding and framework-assisted development.²³¹³ Third, it supports **organizational learning**, because stored traces can later be analyzed to identify recurring insecure prompt patterns, recurring vulnerability classes, and areas where additional policy rules or prompt templates are needed.

The feedback aspect of this layer is equally important. When code is rejected or flagged, the framework should return not only an error state but also a structured explanation to the developer or to the regeneration step. This aligns with the broader secure vibe coding principle that automated tooling should empower the developer with immediate and actionable intelligence rather than produce delayed and opaque security reports.²⁰

4.4.6 Delivery and Integration Layer

The delivery and integration layer connects the framework to the broader software engineering environment. The proposal explicitly states that the framework should be compatible with GitHub, Docker, and standard CI/CD tools, and that later iterations of the prototype will include pluggable validation engines, containerization, and support for multiple analysis tools.¹⁸⁴⁰

This layer matters because Node.js microservice security does not end at source code generation. Microservice systems expose multiple APIs, involve service-to-service communication, and are deployed through containers and pipelines. Research on cloud-native and microservice security consistently identifies the need for security policy enforcement points, API gateways, least privilege, token-based authentication, and container scanning as core parts of a secure architecture.⁴¹³²³³⁴²

SIDF therefore does not operate as an isolated code-generation utility. Instead, it is positioned as an upstream control layer within a secure software delivery pipeline. Its role is to reduce the number of insecure artifacts that reach repository integration, build packaging, or deployment. In practice, this means that only outputs that pass framework validation should move forward to normal repository, testing, and CI/CD stages.

4.5 Core Components of VibeGuard

The prototype implementation of the framework is represented by VibeGuard. While SIDF defines the conceptual model, VibeGuard operationalizes that model through a set of concrete components that collectively implement secure vibe coding for Node.js microservices.⁴²²

4.5.1 Prompt Interceptor

The prompt interceptor is responsible for receiving the developer's raw natural-language request and ensuring that it cannot bypass the framework's control path. This component establishes the boundary between uncontrolled prompting and governed AI-assisted generation. Without this intercept point, it would be impossible to guarantee that prompts are assessed, enriched, or logged consistently.²²

4.5.2 Security-Aware Prompt Enricher

The security-aware prompt enricher transforms a raw request into a constrained request when the task is identified as security-sensitive. For example, if a developer asks for an authentication middleware or a file upload endpoint, the enricher can insert additional instructions requiring secure secret management, path validation, access checks, and safe defaults. This component is central to the proposal's claim that the framework will support security-aware prompt engineering rather than rely solely on post-generation review.⁵²⁹³⁰

4.5.3 Generation Manager

The generation manager submits the enriched prompt to the chosen model and collects the generated artifact. It also manages retries and regeneration when the validation layer rejects a result. This component is especially important because the framework is not based on a single-pass generation assumption. Instead, it assumes that secure output may require iterative refinement, particularly when the first generation contains policy violations or high-severity vulnerabilities.²²¹⁷

4.5.4 Validation Orchestrator

The validation orchestrator coordinates the execution of the configured analysis tools. In the proposed research scope, these tools include ESLint-based security checking and Semgrep-based rule matching, though the architecture is explicitly designed to support pluggable validation mechanisms in later iterations.³⁶⁴⁰ The orchestrator standardizes tool outputs so that the downstream policy engine can reason over them consistently.

4.5.5 Policy and Decision Engine

The policy and decision engine interprets the findings returned by the validation orchestrator and maps them to one of the allowed outcomes: accept, reject and regenerate, or warn and escalate for human review. This component embodies the framework’s governance logic. It is what turns VibeGuard from a passive analyzer into an active development control mechanism. In the context of this thesis, this component is crucial because the research problem is not only the existence of insecure code, but the absence of a structured methodology for deciding how AI outputs should be treated before integration.¹¹⁴³

4.5.6 Provenance Logger

The provenance logger stores all events and artifacts required for auditability, repeatability, and evaluation. The proposal explicitly argues that current workflows lack trust metrics and provenance tracking for AI-generated code and that such omissions make it difficult to scale secure adoption of vibe coding.¹¹²¹ By contrast, VibeGuard retains the coding session as a structured trace, thereby enabling later inspection of how a particular output was generated, what rules were applied, and why a final decision was made.

4.6 Operational Workflow of the Framework

The operational logic of SIDF can be understood as a gated prompt-to-code control loop. This loop consists of six stages: intent capture, prompt classification, prompt enrichment, controlled generation, automated validation, and policy-driven decision and logging.⁵²²²¹

In the first stage, the developer submits a natural-language intent describing the desired functionality. This may include implementing an API endpoint, generating middleware, adding authentication to a service, or creating a secure integration with a database or external service.¹³¹⁴ In the second stage, the prompt intelligence layer classifies the task according to its security relevance. If the request falls into a security-sensitive category, such as authentication, SQL access, secrets, or file operations, the prompt is tagged for enrichment.¹⁸³⁰

In the third stage, the framework enriches the prompt with secure coding constraints and contextual expectations. The enriched request is then forwarded to the model by the generation manager. In the fourth stage, the generated output is immediately passed to the validation orchestrator, which performs static analysis and policy checks using the configured toolchain.²²³⁶³⁷

In the fifth stage, the policy engine decides how the system should respond. If severe findings are detected, the output is rejected and a corrective regeneration step is triggered. If the output is mostly acceptable but still contains cautionary issues, the framework returns warnings and requires explicit developer review. If the output passes the configured checks, it is accepted as approved code.⁵²⁰ In the sixth stage, the entire session is logged into the provenance store and the approved result becomes eligible for repository integration and downstream development tasks.²¹

This workflow operationalizes the secure vibe coding principle advocated in recent literature: the coding loop should not be limited to prompting and output consumption, but should be extended into a tightly coupled prompt-generate-validate process with immediate security feedback.²⁴¹⁷

4.7 Security Integration in Node.js Microservices

A major strength of the proposed solution is that it is designed specifically for **Node.js microservices**, rather than for AI-generated code in the abstract. The proposal explicitly limits the technology scope to JavaScript backend services based on RESTful APIs, SQL databases, and message queues, with a security focus on OWASP Top 10 classes and Node.js-relevant risks such as injection, broken authentication, and data exposure.¹⁴¹⁸

This specialization is justified because microservices introduce architectural properties that affect both generation and validation. They expose many service endpoints, increase the attack surface, and distribute security-relevant logic across authentication services, business services, gateways, middleware, and storage layers. Security research on microservice architectures repeatedly emphasizes that API gateways act as security policy enforcement points and that secure designs must incorporate authentication, authorization, least privilege, and defense in depth across service boundaries.⁴¹⁴⁴³¹³³

The framework supports Node.js microservice security at three levels. At the **generation level**, it constrains prompts and outputs so that code generation is less likely to introduce insecure defaults, unsafe query handling, exposed credentials, or weak token logic.¹⁵⁵ At the **service construction level**, it validates the generated code in the context of common microservice concerns such as input validation, access control boundaries, middleware sequencing, and service API behavior.³⁹²⁵ At the **delivery level**, it aligns with cloud-native best practices such as API gateway enforcement, token-based identity, least privilege, and container scanning, thereby making the proposed framework consistent with secure microservice deployment practices rather than limited to local source code generation alone.³²⁴²³³

This layered treatment of security is important because the proposal explicitly states that existing approaches rarely account for architecture-specific constraints such as microservice patterns,

event-driven design, and message queue integration in Node.js backend systems.⁴⁵ SIDF addresses this gap by designing the solution around those constraints from the beginning, instead of treating architecture as a secondary concern.

4.8 How the Proposed Solution Addresses the Research Problem

The problem addressed by this thesis is not merely that AI systems can generate insecure code. That observation is already well established in the literature and in the proposal. The deeper problem is that there is no comprehensive, security-first workflow for controlling how AI-generated code is produced, assessed, and admitted into development processes for Node.js microservices.⁴⁶⁴³¹²

SIDF addresses this problem in several direct ways. First, it reduces **blind trust in AI output** by intercepting generation before code reaches the developer or repository in uncontrolled form.¹⁰²² Second, it shifts validation from a late-stage or optional activity into a mandatory part of the generation workflow through prompt enrichment and automated analysis.⁵¹¹⁸¹⁷ Third, it introduces **architecture-aware controls** suitable for the realities of Node.js microservice development, including API-centric design, authentication boundaries, and distributed service interfaces.¹⁴¹⁹

Fourth, it generates **measurable evidence of trustworthiness** by logging prompts, outputs, findings, and decisions in a structured way that supports comparative analysis between standard vibe coding and framework-assisted coding.²³²¹ Fifth, it supports **practical adoption** by being designed for integration with the ordinary toolchain of modern software development, including Git workflows, Dockerized services, and CI/CD processes, instead of requiring a separate academic testbed disconnected from practice.¹⁸⁸

For these reasons, the proposed solution does not only mitigate individual vulnerabilities. It also addresses the structural weakness in current AI-assisted development practice, namely the absence of a repeatable, policy-governed method for secure AI code generation in production-relevant software engineering environments.¹¹²¹

4.9 Scientific Contribution and Practical Value

The scientific contribution of the proposed solution lies in the formalization of a **security-integrated architectural framework for vibe coding**. The proposal states that the thesis will contribute by formalizing the security dimension of vibe coding, proposing an empirical methodology for evaluating AI code quality, and highlighting open research questions within a design science software engineering perspective.⁹ SIDF embodies this contribution by defining a conceptual model through which prompt engineering, generation orchestration, automated validation, and governance logging are combined into a single secure development framework.⁴⁵

More specifically, the research contributes four artifacts to knowledge. The first is the **SIDF conceptual framework**, which explains how secure vibe coding should be structured in a software engineering context.⁴ The second is the **VibeGuard prototype**, which operationalizes that framework as a Node.js-based middleware and CLI for real-world coding scenarios.²² The

third is a **generation-time security control pattern**, which integrates prompt enrichment, static analysis, and policy-based gating into the coding loop.⁵¹⁸ The fourth is a **reproducible empirical basis** for evaluating trustworthiness in AI-assisted development through measurable outcomes such as vulnerability density, development time, and test pass rates.²³¹³

The practical value of the solution is equally important. The proposal clearly states that the findings of the thesis are expected to provide actionable value to software developers and teams by offering a security-integrated vibe coding framework and prototype that can accelerate development while systematically reducing vulnerabilities in AI-generated Node.js microservices.⁸ It also states that organizations, architects, and security professionals can use the resulting framework as a reference architecture and policy-oriented guide for integrating AI-generated code into secure CI/CD and governance workflows.²⁶²⁹

In this sense, the proposed solution has both theoretical and applied significance. It contributes a conceptual advancement to the study of secure AI-assisted software engineering while also producing a practical intervention that can be evaluated, refined, and adapted for real-world development practice.

4.10 Chapter Summary

This chapter presented the proposed solution of the thesis, the Security-Integrated Development Framework and its prototype instantiation, VibeGuard, for secure vibe coding in Node.js microservices.⁴⁵ The chapter defined the design goals of the framework, described its layered architecture, explained the role of each major component, and detailed the workflow through which developer prompts are transformed into validated and policy-governed code artifacts.¹²²⁷

The chapter further showed that the proposed solution addresses the identified research problem by embedding security controls directly into the generation workflow, aligning the solution with Node.js microservice realities, and introducing provenance-based governance for AI-generated code.¹²²¹¹⁴ The next chapter will focus on the prototype realization and evaluation of the framework, including the implementation setting, experimental procedure, and empirical assessment of the framework's effectiveness.⁴⁷⁴⁸

Suggested figure placement notes for your document

You can insert these naturally into the chapter:

- **Figure 4.1** Overall architecture of SIDF
Place after Section 4.4 introduction.
- **Figure 4.2** Operational workflow of VibeGuard
Place after Section 4.6.
- **Figure 4.3** Internal component model of VibeGuard
Place after Section 4.5.
- **Figure 4.4** Sequence diagram for secure prompt-to-code execution
Place near the end of Section 4.6.

- **Figure 4.5** Deployment context for SIDF in a Node.js microservice toolchain
Place in Section 4.7 or at the end of Section 4.4.

These figures should make Chapter 4 much stronger because the guideline expects the proposed solution chapter to show the system architecture, component interactions, data flow, and how the solution actually works, not just what it does. Below is an exact figure-by-figure blueprint you can draw directly in Visio, draw.io, Lucidchart, or Word.

General drawing style for all Chapter 4 figures

Use these conventions consistently across all five figures:

- **Rectangle** for process/component
- **Rounded rectangle** for external actor or system
- **Cylinder** for persistent storage
- **Diamond** for decision point
- **Solid arrow** for control or request flow
- **Dashed arrow** for feedback, log transfer, or optional interaction
- **Label every arrow**
- Keep component names consistent throughout the thesis, as required by the guideline.

Recommended color logic:

- Blue: core VibeGuard components
- Orange: external systems
- Green: accepted/approved flow
- Red: rejected/regenerate flow
- Gray: storage/logging/policy artifacts

Figure 4.1 Overall architecture of the Security-Integrated Development Framework

Suggested caption

Figure 4.1: Overall architecture of the Security-Integrated Development Framework (SIDF) and VibeGuard prototype

Purpose of the figure

This figure should present the full conceptual architecture of the proposed solution, showing the major components, data flow, and persistent data elements, which is exactly what the guideline expects in the proposed solution chapter.

Layout

Draw this figure in **three horizontal layers**:

1. **Top layer**: developer and external systems
2. **Middle layer**: VibeGuard core processing pipeline
3. **Bottom layer**: persistent stores and integration targets

Top layer boxes, left to right

1. **Developer**
2. **IDE / CLI Interaction**
3. **LLM Provider API**
4. **Security Analysis Tools**
5. **Git / CI/CD Environment**

You may combine boxes 1 and 2 if you want a cleaner figure, but separating them makes the flow clearer.

Middle layer, inside one large system boundary

Create one large container labeled:

VibeGuard: Security-Integrated Development Middleware

Inside it, place these boxes from left to right:

1. **Prompt Interceptor**
2. **Prompt Risk Classifier**
3. **Security-Aware Prompt Enricher**
4. **Generation Manager**
5. **Validation Orchestrator**
6. **Policy and Decision Engine**
7. **Result Formatter and Feedback Handler**

Bottom layer storage components

Place three cylinders below the VibeGuard container:

1. **Prompt and Output Provenance Log**
2. **Security Policy and Rule Store**
3. **Evaluation Metrics Repository**

Optionally add a fourth if you want more detail:

4. **Prompt Templates and Secure Coding Patterns**

Main arrows, exact flow

Draw these arrows in order:

1. **Developer** → **IDE / CLI Interaction**
Label: *Natural-language development intent*
2. **IDE / CLI Interaction** → **Prompt Interceptor**
Label: *Prompt submission*
3. **Prompt Interceptor** → **Prompt Risk Classifier**
Label: *Captured prompt*
4. **Prompt Risk Classifier** → **Security-Aware Prompt Enricher**
Label: *Security-sensitive prompt*
5. **Prompt Risk Classifier** → **Generation Manager**
Label: *Low-risk prompt*
6. **Security-Aware Prompt Enricher** → **Generation Manager**
Label: *Enriched prompt*
7. **Generation Manager** → **LLM Provider API**
Label: *Code generation request*
8. **LLM Provider API** → **Generation Manager**
Label: *Generated Node.js code*
9. **Generation Manager** → **Validation Orchestrator**
Label: *Generated artifact for validation*
10. **Validation Orchestrator** → **Security Analysis Tools**
Label: *Static analysis / rule execution*
11. **Security Analysis Tools** → **Validation Orchestrator**
Label: *Findings and severity results*
12. **Validation Orchestrator** → **Policy and Decision Engine**
Label: *Normalized findings*
13. **Policy and Decision Engine** → **Result Formatter and Feedback Handler**
Label: *Accept / regenerate / escalate decision*
14. **Result Formatter and Feedback Handler** → **IDE / CLI Interaction**
Label: *Approved code / warning / corrective feedback*
15. **Result Formatter and Feedback Handler** → **Git / CI/CD Environment**
Label: *Approved artifact for integration*

Logging and storage arrows

Use dashed arrows from these boxes downward:

- **Prompt Interceptor** → **Prompt and Output Provenance Log**
Label: *Original prompt*
- **Security-Aware Prompt Enricher** → **Prompt and Output Provenance Log**
Label: *Enriched prompt*
- **Generation Manager** → **Prompt and Output Provenance Log**
Label: *Model metadata and output*

- **Validation Orchestrator** → **Prompt and Output Provenance Log**
Label: *Validation findings*
- **Policy and Decision Engine** → **Prompt and Output Provenance Log**
Label: *Decision trace*
- **Security Policy and Rule Store** → **Prompt Risk Classifier**
Label: *Risk patterns*
- **Security Policy and Rule Store** → **Security-Aware Prompt Enricher**
Label: *Prompt rules*
- **Security Policy and Rule Store** → **Policy and Decision Engine**
Label: *Acceptance policies*
- **Evaluation Metrics Repository** ← **Prompt and Output Provenance Log**
Label: *Experimental measurements*

One-sentence paragraph to use below the figure

Figure 4.1 shows that SIFD places VibeGuard between developer intent, large language model generation, and security validation, thereby transforming direct vibe coding into a governed prompt, generate, validate, and decision workflow.

Figure 4.2 Workflow diagram of secure vibe coding in VibeGuard

Suggested caption

Figure 4.2: Operational workflow of secure vibe coding using VibeGuard

Purpose of the figure

This figure should show the **activity flow** of the framework from prompt submission to final acceptance or regeneration. It visually explains the "how" of the system, which the guideline explicitly asks for.

Best notation

Use a standard **activity diagram** or **flowchart**.

Exact shapes and order

1. **Start**
2. **Receive developer prompt**
3. **Classify prompt risk**
4. **Decision diamond: Is the task security-sensitive?**
5. If **Yes** → **Enrich prompt with security constraints**
6. If **No** → **bypass enrichment**
7. **Send prompt to LLM**
8. **Receive generated code**
9. **Run static analysis and policy checks**

10. **Decision diamond: Are critical vulnerabilities detected?**
11. If **Yes** → **Generate corrective feedback**
12. **Regenerate code with revised prompt**
13. Loop back to **Send prompt to LLM**
14. If **No** → **Decision diamond: Are moderate issues detected?**
15. If **Yes** → **Return warning and require developer review**
16. **Decision diamond: Developer approves?**
17. If **No** → **Revise prompt / regenerate**
18. If **Yes** → **proceed**
19. If no moderate issues → **proceed directly**
20. **Approve artifact**
21. **Log prompt, output, findings, and decision**
22. **Forward approved code to repository / CI workflow**
23. **End**

Exact arrow labels

- Start → Receive developer prompt
Coding request
- Receive developer prompt → Classify prompt risk
Captured intent
- Classify prompt risk → Is the task security-sensitive?
Risk score
- Yes branch → Enrich prompt with security constraints
Sensitive request
- No branch → Send prompt to LLM
Standard request
- Enrich prompt with security constraints → Send prompt to LLM
Protected prompt
- Send prompt to LLM → Receive generated code
Generated output
- Receive generated code → Run static analysis and policy checks
Validation input
- Run static analysis and policy checks → Are critical vulnerabilities detected?
Validation results
- Yes branch → Generate corrective feedback
Critical finding
- Generate corrective feedback → Regenerate code with revised prompt
Remediation request
- Regenerate code with revised prompt → Send prompt to LLM
Re-generation loop
- No branch → Are moderate issues detected?
No critical issues
- Moderate yes → Return warning and require developer review
Review required
- Return warning and require developer review → Developer approves?
Human review

- Approve yes → Approve artifact
Accepted with review
- Approve no → Revise prompt / regenerate
Rejected for revision
- Revise prompt / regenerate → Send prompt to LLM
Retry
- Approve artifact → Log prompt, output, findings, and decision
Accepted result
- Log prompt, output, findings, and decision → Forward approved code to repository / CI workflow
Traceable artifact
- Forward approved code to repository / CI workflow → End
Workflow completed

Short paragraph to use below the figure

Figure 4.2 illustrates the controlled coding lifecycle implemented by VibeGuard, in which prompt classification, security-aware enrichment, automated validation, policy-based decisions, and provenance logging are performed before AI-generated code is accepted into the normal development workflow.

Figure 4.3 Component diagram of VibeGuard

Suggested caption

Figure 4.3: Internal component structure of the VibeGuard prototype

Purpose of the figure

This figure should decompose the prototype into internal modules. The guideline specifically says each architectural component should be discussed separately with reference to the figure showing the architecture.

Layout

Draw one large system boundary labeled:

VibeGuard Prototype

Inside it, arrange components in three columns.

Left column, input-facing components

1. CLI Interface
2. Prompt Interceptor
3. Session Manager

Center column, core intelligence components

4. **Prompt Risk Classifier**
5. **Security-Aware Prompt Enricher**
6. **Generation Manager**
7. **Response Parser**
8. **Validation Orchestrator**
9. **Policy and Decision Engine**
10. **Feedback Generator**

Right column, infrastructure adapters

11. **LLM Connector**
12. **SAST Tool Adapter**
13. **Policy Store Adapter**
14. **Provenance Logger**
15. **Metrics Collector**
16. **Repository / CI Integration Adapter**

External boxes around the system boundary

- **Developer**
- **LLM Provider**
- **Semgrep / ESLint Security Plugins**
- **Git Repository / CI Pipeline**
- **Log and Metrics Storage**

Internal connections

Use solid lines/arrows like this:

- CLI Interface → Prompt Interceptor
- Prompt Interceptor → Session Manager
- Session Manager → Prompt Risk Classifier
- Prompt Risk Classifier → Security-Aware Prompt Enricher
- Security-Aware Prompt Enricher → Generation Manager
- Generation Manager → LLM Connector
- LLM Connector ↔ LLM Provider
- Generation Manager → Response Parser
- Response Parser → Validation Orchestrator
- Validation Orchestrator → SAST Tool Adapter
- SAST Tool Adapter ↔ Semgrep / ESLint Security Plugins
- Validation Orchestrator → Policy and Decision Engine
- Policy and Decision Engine → Feedback Generator
- Feedback Generator → CLI Interface
- Policy and Decision Engine → Repository / CI Integration Adapter
- Repository / CI Integration Adapter ↔ Git Repository / CI Pipeline

- Prompt Interceptor → Provenance Logger
- Security-Aware Prompt Enricher → Provenance Logger
- Generation Manager → Provenance Logger
- Validation Orchestrator → Provenance Logger
- Policy and Decision Engine → Provenance Logger
- Provenance Logger → Log and Metrics Storage
- Metrics Collector → Log and Metrics Storage
- Policy Store Adapter → Policy and Decision Engine
- Policy Store Adapter → Security-Aware Prompt Enricher

Optional labels for interfaces

- Between LLM Connector and LLM Provider: *Model API calls*
- Between SAST Tool Adapter and tools: *Rule execution*
- Between Repository adapter and Git/CI: *Commit / pipeline handoff*
- Between Provenance Logger and storage: *Trace persistence*

Short paragraph to use below the figure

Figure 4.3 refines the architectural view by decomposing VibeGuard into the internal modules responsible for prompt handling, security-aware enrichment, model orchestration, automated validation, policy enforcement, feedback generation, and provenance logging.

Figure 4.4 Sequence diagram for secure prompt-to-code execution

Suggested caption

Figure 4.4: Sequence of interactions in a VibeGuard-assisted coding session

Purpose of the figure

This figure should show one representative end-to-end interaction. The guideline says sequence diagrams are commonly used in this chapter, but only a few should be kept in the main text.

Lifelines, left to right

1. **Developer**
2. **VibeGuard CLI**
3. **Prompt Enricher**
4. **LLM Manager**
5. **LLM API**
6. **Validation Engine**
7. **Policy Engine**
8. **Provenance Log**
9. **Git / CI System**

Exact message sequence

1. Developer → VibeGuard CLI
submitPrompt(taskDescription)
2. VibeGuard CLI → Prompt Enricher
classifyAndEnrich(prompt)
3. Prompt Enricher → VibeGuard CLI
enrichedPrompt / originalPrompt
4. VibeGuard CLI → LLM Manager
generateCode(promptPayload)
5. LLM Manager → LLM API
invokeModel(promptPayload)
6. LLM API → LLM Manager
generatedCode(modelResponse)
7. LLM Manager → Validation Engine
validate(generatedCode)
8. Validation Engine → Validation Engine
runSimgrep
runESLintSecurityChecks
9. Validation Engine → Policy Engine
submitFindings(normalizedResults)
10. Policy Engine → Policy Engine
applyPolicies
11. Policy Engine → Provenance Log
store(prompt, output, findings, decision)

Add alt/decision fragments

Alt fragment 1: Critical findings

- Policy Engine → VibeGuard CLI
rejectAndRegenerate(correctionHints)
- VibeGuard CLI → Prompt Enricher
revisePrompt(correctionHints)
- Prompt Enricher → VibeGuard CLI
revisedPrompt
- VibeGuard CLI → LLM Manager
generateCode(revisedPrompt)

You can stop the redraw here with a note:

Loop until policy-compliant or max retry threshold reached

Alt fragment 2: Accepted output

- Policy Engine → VibeGuard CLI
approve(acceptedCode)

- VibeGuard CLI → Git / CI System
forwardApprovedArtifact(code, metadata)
- VibeGuard CLI → Developer
returnApprovedCode

Alt fragment 3: Warning with human review

- Policy Engine → VibeGuard CLI
warn(reviewRequired, findings)
- VibeGuard CLI → Developer
displayWarnings(findings)
- Developer → VibeGuard CLI
approveOrReject

If approved:

- VibeGuard CLI → Git / CI System
forwardApprovedArtifact

If rejected:

- VibeGuard CLI → Prompt Enricher
revisePrompt

Short paragraph to use below the figure

Figure 4.4 shows that VibeGuard does not treat code generation as a single transaction. Instead, generation, validation, policy enforcement, and provenance logging are part of a controlled interaction sequence that may include regeneration or explicit human review before acceptance.

Figure 4.5 Deployment diagram of SIDF in a Node.js microservice toolchain

Suggested caption

Figure 4.5: Deployment context of SIDF and VibeGuard in a Node.js microservice development environment

Purpose of the figure

This figure connects the framework to the real engineering environment. It is very useful because your proposal explicitly targets integration with GitHub, Docker, and standard CI/CD tools, and the research scope is specifically Node.js microservices with REST APIs, message queues, and SQL databases.

Main deployment nodes

Draw these as 3D deployment nodes or nested rectangles.

Node 1: Developer Workstation

Inside this node:

- **IDE / Terminal**
- **VibeGuard CLI**
- **Local Project Workspace**
- **Docker Client**

Node 2: External AI Service

Inside this node:

- **LLM Provider API**

Node 3: Version Control and Pipeline Platform

Inside this node:

- **Git Repository**
- **CI/CD Runner**
- **Container Registry**

Node 4: Cloud / Server Runtime Environment

Inside this node:

- **API Gateway**
- **Node.js Auth Service**
- **Node.js Business Service**
- **Node.js File Service**
- **Message Broker**
- **SQL Database**
- **Monitoring / Logging Service**

Exact deployment connections

1. **IDE / Terminal ↔ VibeGuard CLI**
Label: *Developer interaction*
2. **VibeGuard CLI ↔ Local Project Workspace**
Label: *Read / write generated code*
3. **VibeGuard CLI ↔ LLM Provider API**
Label: *Prompt and generation calls*

4. **VibeGuard CLI ↔ Git Repository**
Label: *Approved artifact commit / push*
5. **Git Repository → CI/CD Runner**
Label: *Pipeline trigger*
6. **CI/CD Runner → Container Registry**
Label: *Build and publish image*
7. **CI/CD Runner → API Gateway**
Label: *Deploy service revision*
8. **API Gateway → Node.js Auth Service**
Label: *Routed secure request*
9. **API Gateway → Node.js Business Service**
Label: *Routed business request*
10. **API Gateway → Node.js File Service**
Label: *Routed file operation request*
11. **Node.js Business Service ↔ SQL Database**
Label: *Parameterized data access*
12. **Node.js Business Service ↔ Message Broker**
Label: *Event publish / consume*
13. **Node.js Auth Service ↔ SQL Database**
Label: *Identity data access*
14. **All Node.js services → Monitoring / Logging Service**
Label: *Operational logs and metrics*
15. Optional dashed arrow from **VibeGuard CLI** to **Monitoring / Logging Service**
Label: *Generation provenance export*

Security annotations to place on the deployment figure

Add small notes near the relevant nodes:

- Near **API Gateway**: *Policy enforcement point*
- Near **Auth Service**: *OAuth2 / token-based identity*
- Near service nodes: *Least privilege access*
- Near pipeline/registry: *Container and deployment security checks*
- Near VibeGuard CLI: *Generation-time validation and provenance logging*

Short paragraph to use below the figure

Figure 4.5 situates VibeGuard within the broader Node.js microservice toolchain, showing that the framework operates as an upstream security control before source code enters repository, build, containerization, and deployment stages.

Recommended order of insertion in Chapter 4

Insert the figures in this order:

- **Figure 4.1** after Section 4.4 introduction

- **Figure 4.2** inside Section 4.6
- **Figure 4.3** inside Section 4.5
- **Figure 4.4** near the end of Section 4.6
- **Figure 4.5** inside Section 4.7

That order moves from macro architecture to workflow, internal structure, interaction sequence, and deployment context, which reads naturally in a thesis.

Ready-to-use in-text references

You can paste these sentences into Chapter 4:

- **For Figure 4.1:**
Figure 4.1 presents the overall architecture of SIDS and shows how VibeGuard mediates the interaction between developer intent, LLM-based code generation, automated security validation, and downstream integration workflows.
- **For Figure 4.2:**
Figure 4.2 illustrates the operational workflow of secure vibe coding in VibeGuard, from prompt submission and risk classification to validation, decision making, and provenance logging.
- **For Figure 4.3:**
Figure 4.3 decomposes VibeGuard into its major internal components, highlighting the modules responsible for prompt processing, security-aware enrichment, orchestration, validation, policy enforcement, and traceability.
- **For Figure 4.4:**
Figure 4.4 shows the interaction sequence of a typical VibeGuard-assisted coding session, including generation, validation, policy evaluation, regeneration, and final acceptance.
- **For Figure 4.5:**
Figure 4.5 places SIDS within a practical Node.js microservice environment and shows how generation-time controls connect with repository, pipeline, container, gateway, and service deployment stages.

Draw.io ready prompt

The Mermaid blocks below are aligned with the university expectation that Chapter 4 should show the proposed solution through architecture, component interaction, data flow, and operational behavior, and they preserve the SIDS and VibeGuard structure defined in your proposal.

Use one Mermaid block per figure. Import them separately into draw.io so layout adjustment is easier. After import, you can fine-tune spacing, colors, and line routing manually.

Figure 4.1 Mermaid code

Caption: Figure 4.1: Overall architecture of the Security-Integrated Development Framework (SIDF) and VibeGuard prototype

This figure reflects the proposal's model in which VibeGuard sits between developer prompts, LLM generation, validation tools, policy enforcement, and provenance logging.

```
flowchart TB
    %% Top layer
    DEV([Developer])
    IDE([IDE / CLI Interaction])
    LLM([LLM Provider API])
    SAST([Security Analysis Tools])
    CICD([Git / CI/CD Environment])

    %% Core system
    subgraph VG["VibeGuard: Security-Integrated Development Middleware"]
        direction LR
        PI[Prompt Interceptor]
        PRC[Prompt Risk Classifier]
        SPA[Security-Aware Prompt Enricher]
        GM[Generation Manager]
        VO[Validation Orchestrator]
        PDE[Policy and Decision Engine]
        RF[Result Formatter and Feedback Handler]
    end

    %% Storage layer
    PLOG[(Prompt and Output Provenance Log)]
    RULES[(Security Policy and Rule Store)]
    METRICS[(Evaluation Metrics Repository)]
    TEMPLATES[(Prompt Templates and Secure Coding Patterns)]

    %% Main flow
    DEV -->|Natural-language development intent| IDE
    IDE -->|Prompt submission| PI
    PI -->|Captured prompt| PRC
    PRC -->|Security-sensitive prompt| SPA
    PRC -->|Low-risk prompt| GM
    SPA -->|Enriched prompt| GM

    GM -->|Code generation request| LLM
    LLM -->|Generated Node.js code| GM

    GM -->|Generated artifact for validation| VO
    VO -->|Static analysis / rule execution| SAST
    SAST -->|Findings and severity results| VO
    VO -->|Normalized findings| PDE
    PDE -->|Accept / regenerate / escalate decision| RF

    RF -->|Approved code / warning / corrective feedback| IDE
    RF -->|Approved artifact for integration| CICD

    %% Logging and policy links
    PI -.->|Original prompt| PLOG
    SPA -.->|Enriched prompt| PLOG
    GM -.->|Model metadata and output| PLOG
    VO -.->|Validation findings| PLOG
```

```

PDE -->|Decision trace| PLOG

RULES -->|Risk patterns| PRC
RULES -->|Prompt rules| SPA
RULES -->|Acceptance policies| PDE
TEMPLATES -->|Secure coding patterns| SPA

PLOG -->|Experimental measurements| METRICS

%% Styling
classDef ext fill:#ffe6cc,stroke:#d79b00,color:#000,stroke-width:1px;
classDef core fill:#dae8fc,stroke:#6c8ebf,color:#000,stroke-width:1px;
classDef data fill:#f5f5f5,stroke:#666,color:#000,stroke-width:1px;

class DEV,IDE,LLM,SAST,CICD ext;
class PI,PRC,SPA,GM,VO,PDE,RF core;
class PLOG,RULES,METRICS,TEMPLATES data;

```

Figure 4.2 Mermaid code

Caption: Figure 4.2: Operational workflow of secure vibe coding using VibeGuard
This workflow represents the controlled prompt, generate, validate, decide, and log cycle proposed for VibeGuard-assisted coding.

```

flowchart TD
    A([Start]) --> B[Receive developer prompt]
    B --> C[Classify prompt risk]
    C --> D{Is the task security-sensitive?}

    D -- Yes --> E[Enrich prompt with security constraints]
    D -- No --> G[Send prompt to LLM]

    E --> G
    G --> H[Receive generated code]
    H --> I[Run static analysis and policy checks]
    I --> J{Are critical vulnerabilities detected?}

    J -- Yes --> K[Generate corrective feedback]
    K --> L[Regenerate code with revised prompt]
    L --> G

    J -- No --> M{Are moderate issues detected?}
    M -- No --> Q[Approve artifact]
    M -- Yes --> N[Return warning and require developer review]
    N --> O{Developer approves?}

    O -- Yes --> Q
    O -- No --> P[Revise prompt / regenerate]
    P --> G

    Q --> R[Log prompt, output, findings, and decision]
    R --> S[Forward approved code to repository / CI workflow]
    S --> T([End])

```

```

classDef startend fill:#d5e8d4,stroke:#82b366,color:#000,stroke-
width:1px;
classDef process fill:#dae8fc,stroke:#6c8ebf,color:#000,stroke-width:1px;
classDef decision fill:#fff2cc,stroke:#d6b656,color:#000,stroke-
width:1px;
classDef warning fill:#f8cecc,stroke:#b85450,color:#000,stroke-width:1px;

class A,T startend;
class B,C,E,G,H,I,K,L,N,P,Q,R,S process;
class D,J,M,O decision;

```

Figure 4.3 Mermaid code

Caption: Figure 4.3: Internal component structure of the VibeGuard prototype

This component view decomposes VibeGuard into prompt handling, enrichment, orchestration, validation, policy enforcement, integration, and provenance modules, consistent with the architecture discussed in Chapter 4.

```

flowchart LR
    DEV([Developer])
    LLMEXT([LLM Provider])
    SASTEXT([Semgrep / ESLint Security Plugins])
    GITEXT([Git Repository / CI Pipeline])
    STORE([Log and Metrics Storage])

    subgraph VG["VibeGuard Prototype"]
        direction LR

        subgraph INP["Input-Facing Components"]
            direction TB
            CLI[CLI Interface]
            PIN[Prompt Interceptor]
            SM[Session Manager]
        end

        subgraph CORE["Core Intelligence Components"]
            direction TB
            PRC[Prompt Risk Classifier]
            SPA[Security-Aware Prompt Enricher]
            GM[Generation Manager]
            RP[Response Parser]
            VO[Validation Orchestrator]
            PDE[Policy and Decision Engine]
            FG[Feedback Generator]
        end

        subgraph INFRA["Infrastructure Adapters"]
            direction TB
            LLMC[LLM Connector]
            SASTA[SAST Tool Adapter]
            PSA[Policy Store Adapter]
            PL[Provenance Logger]
            MC[Metrics Collector]
            RCIA[Repository / CI Integration Adapter]
        end
    end

```

```

end

POLICY[(Security Policy Store)]

DEV -->|Prompt / commands| CLI
CLI --> PIN
PIN --> SM
SM --> PRC
PRC --> SPA
SPA --> GM
GM --> LLMC
LLMC -->|Model API calls| LLMEXT
LLMEXT -->|Generated response| LLMC
LLMC --> GM
GM --> RP
RP --> VO
VO --> SASTA
SASTA -->|Rule execution| SASTEXT
SASTEXT -->|Findings| SASTA
SASTA --> VO
VO --> PDE
PDE --> FG
FG --> CLI
PDE --> RCIA
RCIA -->|Commit / pipeline handoff| GITEXT

PIN -. -> PL
SPA -. -> PL
GM -. -> PL
VO -. -> PL
PDE -. -> PL
PL -->|Trace persistence| STORE
MC -->|Metrics persistence| STORE

PSA --> PDE
PSA --> SPA
POLICY --> PSA

classDef ext fill:#ffe6cc,stroke:#d79b00,color:#000,stroke-width:1px;
classDef core fill:#dae8fc,stroke:#6c8ebf,color:#000,stroke-width:1px;
classDef infra fill:#e1d5e7,stroke:#9673a6,color:#000,stroke-width:1px;
classDef data fill:#f5f5f5,stroke:#666,color:#000,stroke-width:1px;

class DEV,LLMEXT,SASTEXT,GITEXT ext;
class CLI,PIN,SM,PRC,SPA,GM,RP,VO,PDE,FG core;
class LLMC,SASTA,PSA,PL,MC,RCIA infra;
class STORE,POLICY data;

```

Figure 4.4 Mermaid code

Caption: Figure 4.4: Sequence of interactions in a VibeGuard-assisted coding session

This sequence diagram shows that generation is not treated as a single isolated transaction, but as a controlled interaction involving enrichment, validation, policy evaluation, regeneration if needed, and provenance logging.

```

sequenceDiagram
    autonumber
    actor Developer
    participant CLI as VibeGuard CLI
    participant Enricher as Prompt Enricher
    participant LLManager as LLM Manager
    participant LLMAPI as LLM API
    participant Validator as Validation Engine
    participant Policy as Policy Engine
    participant Log as Provenance Log
    participant GitCI as Git / CI System

    Developer->>CLI: submitPrompt(taskDescription)
    CLI->>Enricher: classifyAndEnrich(prompt)
    Enricher-->>CLI: enrichedPrompt / originalPrompt
    CLI->>LLManager: generateCode(promptPayload)
    LLManager->>LLMAPI: invokeModel(promptPayload)
    LLMAPI-->>LLManager: generatedCode(modelResponse)
    LLManager->>Validator: validate(generatedCode)

    Validator->>Validator: runSemgrep
    Validator->>Validator: runESLintSecurityChecks
    Validator-->>Policy: submitFindings(normalizedResults)
    Policy->>Policy: applyPolicies
    Policy->>Log: store(prompt, output, findings, decision)

    alt Critical findings detected
        Policy-->>CLI: rejectAndRegenerate(correctionHints)
        CLI->>Enricher: revisePrompt(correctionHints)
        Enricher-->>CLI: revisedPrompt
        loop Regenerate until compliant or retry threshold reached
            CLI->>LLManager: generateCode(revisedPrompt)
            LLManager->>LLMAPI: invokeModel(revisedPrompt)
            LLMAPI-->>LLManager: regeneratedCode
            LLManager->>Validator: validate(regeneratedCode)
            Validator-->>Policy: submitFindings(normalizedResults)
            Policy->>Log: store(revisedPrompt, regeneratedCode, findings,
decision)
        end
    end
    else Accepted output
        Policy-->>CLI: approve(acceptedCode)
        CLI->>GitCI: forwardApprovedArtifact(code, metadata)
        CLI-->>Developer: returnApprovedCode
    end
    else Warning with human review
        Policy-->>CLI: warn(reviewRequired, findings)
        CLI-->>Developer: displayWarnings(findings)
        Developer->>CLI: approveOrReject
        alt Developer approves
            CLI->>GitCI: forwardApprovedArtifact(code, metadata)
            CLI->>Log: store(userApprovalDecision)
        end
        else Developer rejects
            CLI->>Enricher: revisePrompt
            Enricher-->>CLI: revisedPrompt
        end
    end
end

```

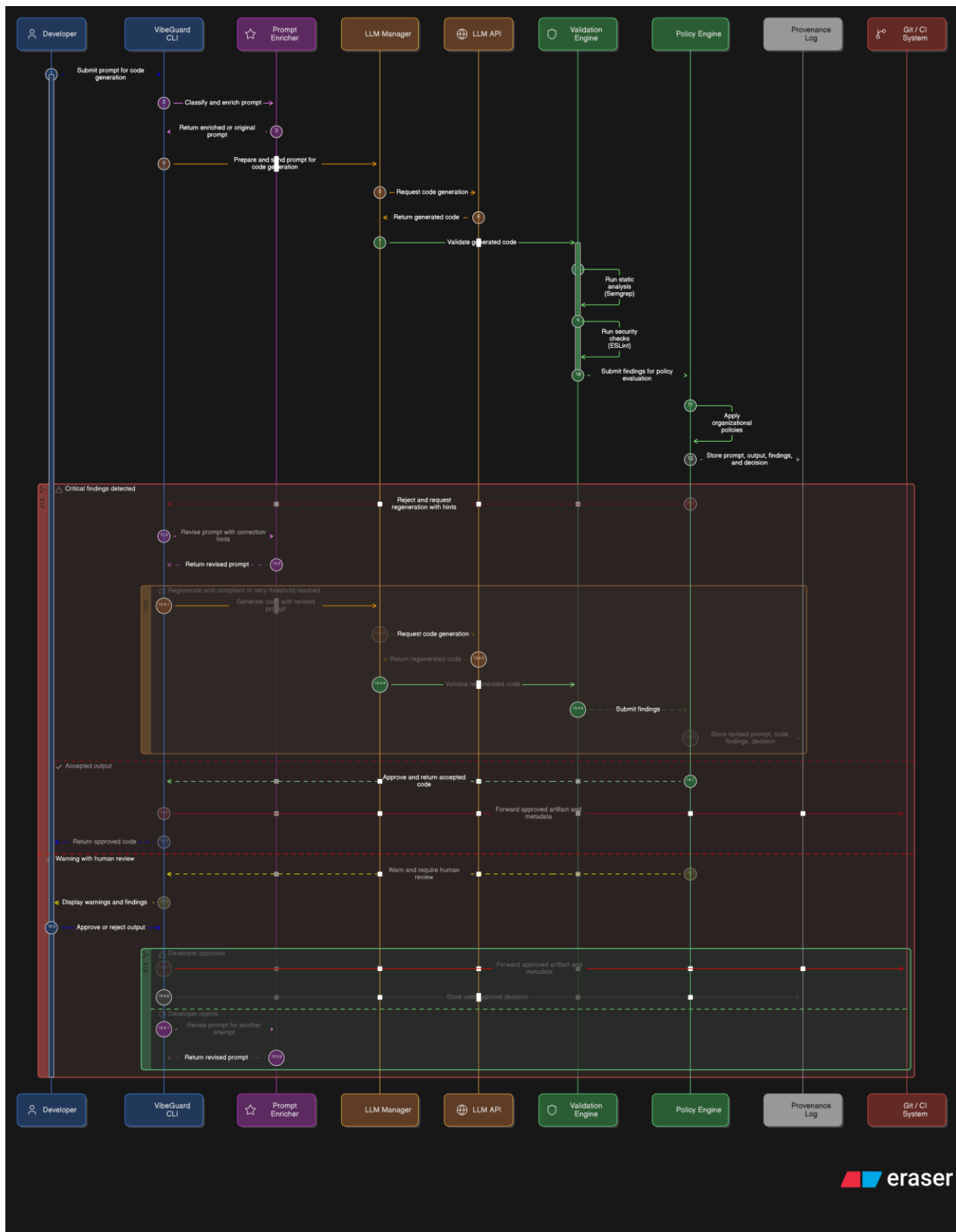


Figure 4.5 Mermaid code

Caption: Figure 4.5: Deployment context of SIDF and VibeGuard in a Node.js microservice development environment

This deployment-oriented figure places VibeGuard in the broader toolchain envisioned by the proposal, including Git integration, Dockerized delivery, CI/CD, API gateway routing, and Node.js microservices with databases and message brokers.

```
flowchart LR
    subgraph WS["Developer Workstation"]
        direction TB
        IDE[IDE / Terminal]
        VGCLI[VibeGuard CLI]
        LWS[Local Project Workspace]
        DOCKER[Docker Client]
    end

    subgraph AI["External AI Service"]
        direction TB
        LLMAPI[LLM Provider API]
    end

    subgraph VC["Version Control and Pipeline Platform"]
        direction TB
        GIT[Git Repository]
        RUNNER[CI / CD Runner]
        REG[Container Registry]
    end

    subgraph CLOUD["Cloud / Server Runtime Environment"]
        direction TB

        APIGW[API Gateway]
        AUTH[Node.js Auth Service]
        BIZ[Node.js Business Service]
        FILES[Node.js File Service]
        MQ[Message Broker]
        DB[(SQL Database)]
        MON[(Monitoring / Logging Service)]
    end

    %% Local interactions
    IDE -->|Developer interaction| VGCLI
    VGCLI -->|Read / write generated code| LWS
    VGCLI -->|Build / package support| DOCKER

    %% AI interaction
    VGCLI -->|Prompt and generation calls| LLMAPI
    LLMAPI -->|Generated code responses| VGCLI

    %% Version control and pipeline
    VGCLI -->|Approved artifact commit / push| GIT
    GIT -->|Pipeline trigger| RUNNER
    RUNNER -->|Build and publish image| REG
    RUNNER -->|Deploy service revision| APIGW

    %% Gateway routing
```

```

APIGW -->|Routed secure request| AUTH
APIGW -->|Routed business request| BIZ
APIGW -->|Routed file operation request| FILES

%% Service dependencies
AUTH -->|Identity data access| DB
BIZ -->|Parameterized data access| DB
BIZ -->|Event publish / consume| MQ
FILES -->|Metadata and storage access| DB

%% Observability
AUTH -->|Operational logs and metrics| MON
BIZ -->|Operational logs and metrics| MON
FILES -->|Operational logs and metrics| MON
VGCLI -->|Generation provenance export| MON

%% Security notes
NOTE1[Policy enforcement point]
NOTE2[OAuth2 / token-based identity]
NOTE3[Least privilege access]
NOTE4[Container and deployment security checks]
NOTE5[Generation-time validation and provenance logging]

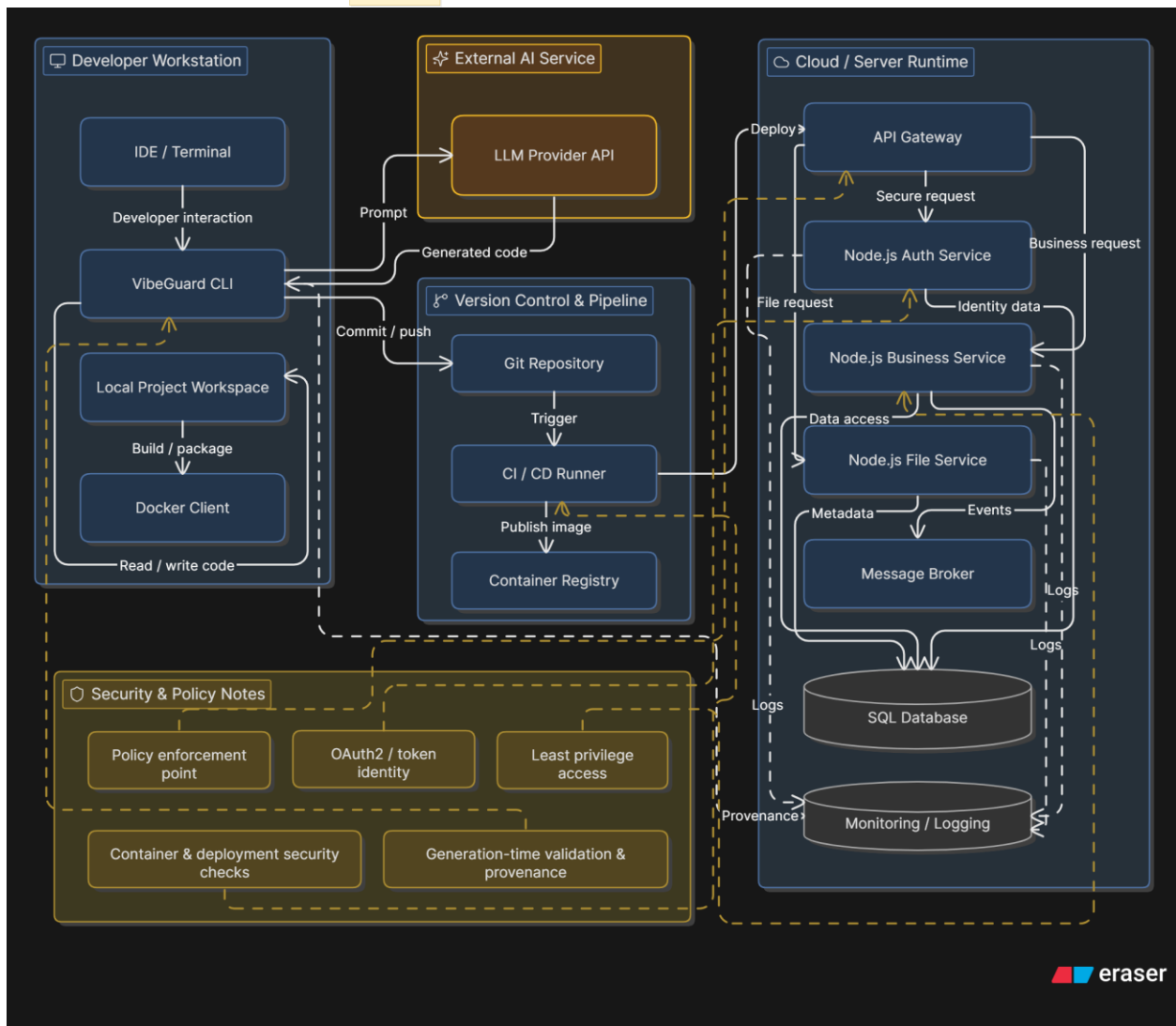
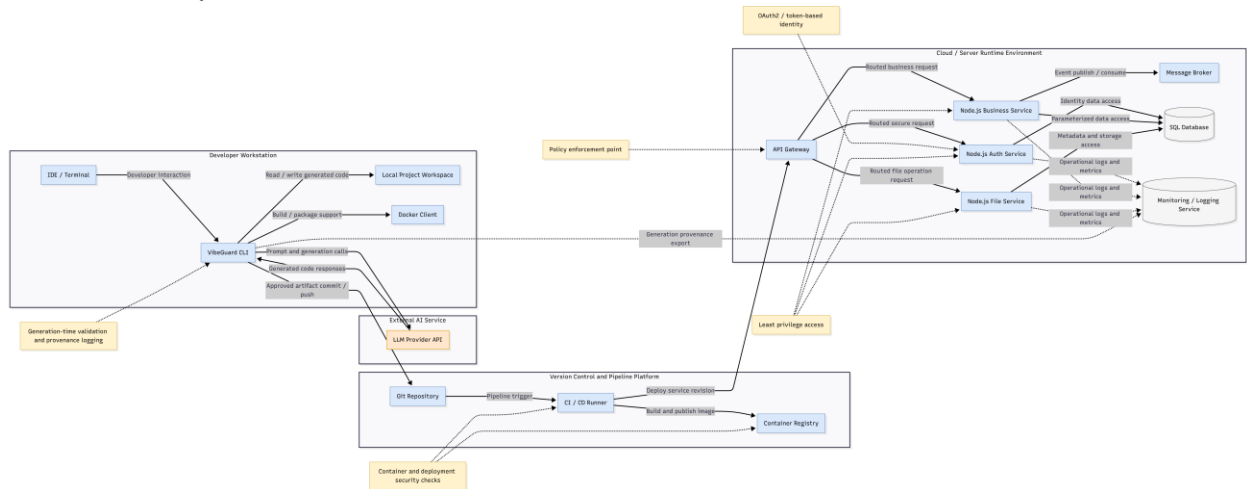
NOTE1 --> APIGW
NOTE2 --> AUTH
NOTE3 --> AUTH
NOTE3 --> BIZ
NOTE3 --> FILES
NOTE4 --> RUNNER
NOTE4 --> REG
NOTE5 --> VGCLI

classDef node fill:#dae8fc,stroke:#6c8ebf,color:#000,stroke-width:1px;
classDef ext fill:#ffe6cc,stroke:#d79b00,color:#000,stroke-width:1px;
classDef data fill:#f5f5f5,stroke:#666,color:#000,stroke-width:1px;
classDef note fill:#fff2cc,stroke:#d6b656,color:#000,stroke-width:1px;

class IDE,VGCLI,LWS,DOCKER,GIT,RUNNER,REG,APIGW,AUTH,BIZ,FILES,MQ node;
class LLMAPI ext;
class DB,MON data;
class NOTE1,NOTE2,NOTE3,NOTE4,NOTE5 note;

```

Mermaid.ai 



Recommended usage notes

- Use **Figure 4.1** and **Figure 4.2** exactly as they are, they are the most important for Chapter 4.
- If draw.io compresses **Figure 4.3**, import it first, then widen the canvas and reapply auto-layout.
- **Figure 4.4** is usually best kept in portrait width.
- **Figure 4.5** is the one most likely to need manual spacing after import because deployment figures are naturally wider.

Safe minor edits you can make after import

You can safely do these without changing the meaning:

- shorten “Security-Aware Prompt Enricher” to “Prompt Enricher”
- shorten “Policy and Decision Engine” to “Policy Engine”
- change “Git / CI/CD Environment” to “Repository and CI/CD”
- change “LLM Provider API” to a specific model provider only if you later decide to name one in the thesis

You should **not** change the core logic that VibeGuard intercepts prompts, enriches risky tasks, validates outputs before acceptance, and logs full provenance, because that is central to the proposal and the chapter argument.

If you want, I can next produce a second pass with:

1. **simplified Mermaid versions** for cleaner thesis visuals, or
2. **figure-by-figure caption text and paragraph explanations** ready to paste under each figure.

References

- 1, 2, 3, 6, 27, 47, 48. [Missing authors]. (n.d.). Department of Computer Science - Brief History.
- 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 18, 21, 22, 23, 26, 29, 34, 35, 36, 40, 43, 45, 46. Hailu, Fantahun Bishaw. (n.d.). Design and Evaluation of a Security-Integrated Vibe Coding Framework for Node.js Microservices..
- 16, 17, 20, 24, 28, 30. Ge, Yuyao et al.. (2025). A Survey of Vibe Coding with Large Language Models. <http://arxiv.org/abs/2510.12399v2>
- 19, 25, 33, 42. Rahaman, Md Shahidur et al.. (2023). Static-Analysis-Based Solutions to Security Challenges in Cloud-Native Systems: Systematic Mapping Study. *Sensors*, 23(4), 1755. <https://doi.org/10.3390/s23041755>
- 31, 32, 41, 44. Rahaman, Md Shahidur et al.. (2023). Access Control Design Practice and Solutions in Cloud-Native Architecture: A Systematic Mapping Study. <https://doi.org/10.3390/s23073413>
- 37, 38, 39. Mohsin, Ahmad et al.. (n.d.). Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs. <http://arxiv.org/abs/2406.12513v1>

5. Experimentation/Prototype and Evaluation

Experimentation / prototype and evaluation content will be here

6. Conclusions

Conclusion content will be here.

References

- [1] A. Karpathy, "There's a new kind of coding I call 'vibe coding,'" Twitter/X, Feb. 3, 2025. [Online]. Available: <https://x.com/karpathy/status/1886192184808149383>, Last accessed on Nov. 14, 2025].
- [2] J. Hueckelheim, V. Sathia, and S. B. Qian, "Data race detection through vibe translation," in *Proc. SC'25 Workshops of the Int. Conf. for High Performance Computing, Networking, Storage— and Analysis*, Denver, CO, USA, 2025, pp. 198–206.
- [3] C. Meske, T. Hermanns, E. von der Weiden, K.-U. Loser, and T. Berger, "Vibe coding as a reconfiguration of intent mediation in software development: Definition, implications, and research agenda," *arXiv preprint arXiv:2507.21928*, 2025.
- [4] Partnership on AI, "Human-AI collaboration framework & case studies," San Francisco, CA, USA, White Paper, Mar. 2025. [Online]. Available: <https://www.partnershiponai.org>, Last accessed on Nov. 14, 2025].
- [5] A. Sarkar and I. Drosos, "Vibe coding: Programming through conversation with artificial intelligence," *arXiv preprint arXiv:2506.23253*, 2025.
- [6] F. Pesce and W. Cheungpasitporn, "Vibe coding in nephrology education: Clinician-led, AI-assisted development of open-source interactive learning tools," *Renal Failure*, vol. 47, no. 1, p. e14159, 2025.
- [7] M. Chow and O. Ng, "From technology adopters to creators: Leveraging AI-assisted vibe coding to transform clinical teaching and learning," *Medical Teacher*, vol. 47, no. 1, pp. 1–18, Jan. 2025.
- [8] F. Geng et al., "Exploring student-AI interactions in vibe coding," *arXiv preprint arXiv:2507.22614*, 2025.
- [9] P. P. Ray, "A review on vibe coding: Fundamentals, state-of-the-art, challenges and future directions," *Authorea Preprints*, 2025.

- [10] J. Mitchell and Y. Shaaban, "Position: Vibe coding needs vibe reasoning: Improving vibe coding with formal verification," in *Proc. 1st ACM SIGPLAN Int. Workshop on Language Models and Programming Languages (LLMPL '25)*, San Diego, CA, USA, 2025, pp. 84–90.
- [11] T. T. Bannon and P. Laplante, "Generative AI in the software development lifecycle," *Computer*, vol. 57, no. 12, pp. 27–34, Dec. 2024.
- [12] G. Adolphs et al., "The hidden risks of LLM-generated web application code," *arXiv preprint arXiv:2504.20612*, Apr. 2025.
- [13] Cloud Security Alliance, "Understanding security risks in AI-generated code," CSA Blog, Jul. 2025. [Online]. Available: <https://cloudsecurityalliance.org/blog/2025/07/09/understanding-security-risks-in-ai-generated-code>, last accessed on Nov. 14, 2025.
- [14] Center for Security and Emerging Technology (CSET), "Cybersecurity risks of AI-generated code," Georgetown Univ., Washington, DC, USA, Tech. Rep., 2025. [Online]. Available: <https://cset.georgetown.edu>, last accessed on Nov. 14, 2025.
- [15] J. Cito and D. Bork, "Lost in code generation: Reimagining the role of software models in AI-driven software engineering," *arXiv preprint arXiv:2511.02475*, 2025.
- [16] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI," *arXiv preprint arXiv:2505.19443*, 2025.
- [17] D. Narayan et al., "Securing software development: Integrating security into CI/CD pipelines," in *Proc. IEEE Secure Development Conf. (SecDev '24)*, 2024, pp. 41–50.
- [18] A. Awad and L. Shaker, "DevSecOps: A systematic review of security practices in continuous development," *IEEE Trans. Software Eng.*, vol. 48, no. 3, pp. 812–832, Mar. 2022.

Annexes

Annex A: Title of the annex

Description to it

Annex B: Title of the annex

Description to it

I, the undersigned, declare that this thesis is my original work and has not been presented for a degree in any other university, and that all source of materials used for the thesis have been duly acknowledged

Declared by:

Name: _____

Signature: _____

Date: _____

Confirmed by Advisor:

Name: _____

Signature: _____

Date: _____